

Programsko inženjerstvo ak.god 2025./2026.

Sveučilište u Zagrebu

Fakultet elektrotehnike i računarstva

Mindfulness - Platforma za mentalno zdravlje

Tim: <TG07.4>

i na kvadrat

Članovi

1. Fran Pilica - voditelj, backend
2. Karlo Ivančić - backend
3. Ana Smoljo - dokumentacija, backend
4. Lovre Baus - frontend
5. Elizej Kojić - frontend
6. Ivan Lukić - frontend

Nastavnik: Vlado Sruk

Cilj projektnog zadatka

Cilj ovog studentskog projekta je osmisliti i razviti platformu (Mindfulness) koja potiče korisnike na brigu o mentalnom zdravlju, razvija navike svjesnog življenja te ugodan način za unapređenje blagostanja.

Problematika zadatka

U današnje se vrijeme sve veći broj ljudi suočava s problemima stresa, anksioznosti, nesanice i smanjenog fokusa uzrokovanih brzim tempom života, preopterećenjem informacijama i manjkom brige o mentalnom zdravlju. Unatoč porastu svijesti o važnosti mentalnog zdravlja, mnogi pojedinci nemaju pristup kvalitetnim alatima i resursima koji bi im pomogli razviti zdrave navike i održavati emocionalnu ravnotežu.

Trenutno dostupne aplikacije za mentalno zdravlje često su fragmentirane - nude ograničen sadržaj, nedovoljnu personalizaciju i ne povezuju različite aspekte korisnikova psihofizičkog stanja (poput sna, fizičke aktivnosti, stresa i fokusa). Korisnici moraju koristiti više različitih aplikacija za meditaciju, praćenje navika i planiranje aktivnosti, što otežava održavanje kontinuiteta i motivacije.

Nadalje, nedostatak individualiziranih preporuke dovodi do toga da korisnici ne dobivaju sadržaj prilagođen njihovim potrebama i ciljevima. Mnogi odustaju od aplikacije jer ne vide konkretne rezultate ili ne znaju kako pravilno pristupiti vježbama.

Cilj platforme Mindfulness riješiti je navedene probleme razvojem integrirane platforme koja omogućuje praćenje raspoloženja i navika, vođene meditacije, personalizirane planove i preporuke te podršku stručnjaka. Aplikacija će omogućiti jedinstveno, personalizirano i sigurno okruženje koje potiče redovitu brigu o mentalnom zdravlju i dugoročno unapređuje kvalitetu života korisnika.

Korisnički zahtjevi

Naša će aplikacija biti usmjerena na više različitih grupa korisnika, s prikladnim funkcionalnostima.

Korisnici će moći putem aplikacije voditi dnevnu evidenciju raspoloženja, sudjelovati u izazovima i primati preporuke. Osim toga korisnik može pratiti i kvalitetu sna, razinu stresa, fokus, unos kofeina te fizičku aktivnost. Korisnik može zakazivati termine prakse te ih sinkrozinirati s vanjskim kalendarom. Također će imati opciju subskripcije na obavijesti putem push notifikacija ili e-maila.

Treneri i/ili terapeuti putem aplikacije će imati mogućnost objavljivanja programa i planova te praćenja napretka korisnika, uz korisnikovo dopuštenje.

Aministratori će nadgledati sigurnost podataka, dodavati i uređivati sadržaj, pratiti statistiku te osigurati nesmetan rad aplikacije.

Potencijalna korist projekta

Aplikacija za mentalno zdravlje može imati značajan pozitivan utjecaj na pojedinca i zajednicu. Našim korisnicima omogućujemo razvoj navika/rutine, smanjenje tjeskobe te poboljšanje kvalitete sna i fokusa. Kroz vođene vježbe disanja i meditacije korisnici stječu praktične alate za suočavanje sa svakodnevnim izazovima i stresnim situacijama. Praćenjem raspoloženja korisnici stječu uvid u vlastite obrasce ponašanja te ih mogu promijeniti/poboljšati.

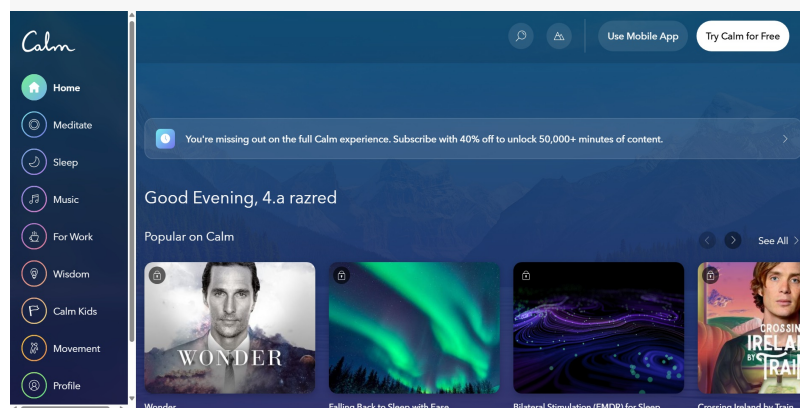
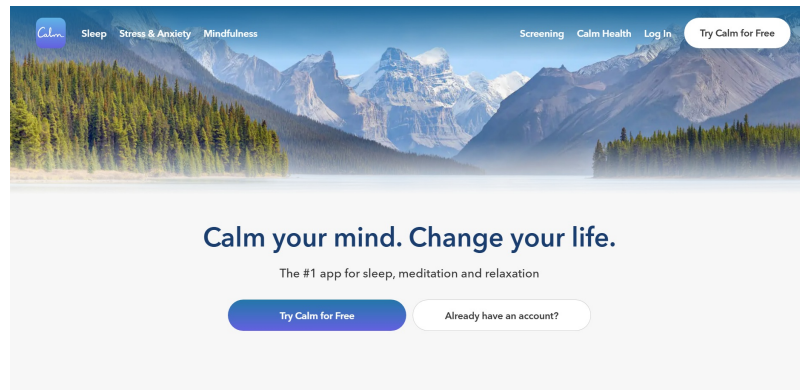
Projekt također nudi potencijalnu korsit za trenere i terapeute. Uz pristanak korisnika, treneri i terapeuti imaju pristup podacima o napretku i raspoloženju, što bi im omogućilo objavljivanje još personaliziranih sadržaja i izradu učinkovitijih planova treninga.

Ova aplikacija može povećati svijest o važnosti samopomoći, mentalnom zdravlju te razvoju zdravih životnih navika. Naša platforma može doprinijeti destigmatizaciji mentalnog zdravlja.

Postojeća slična rješenja

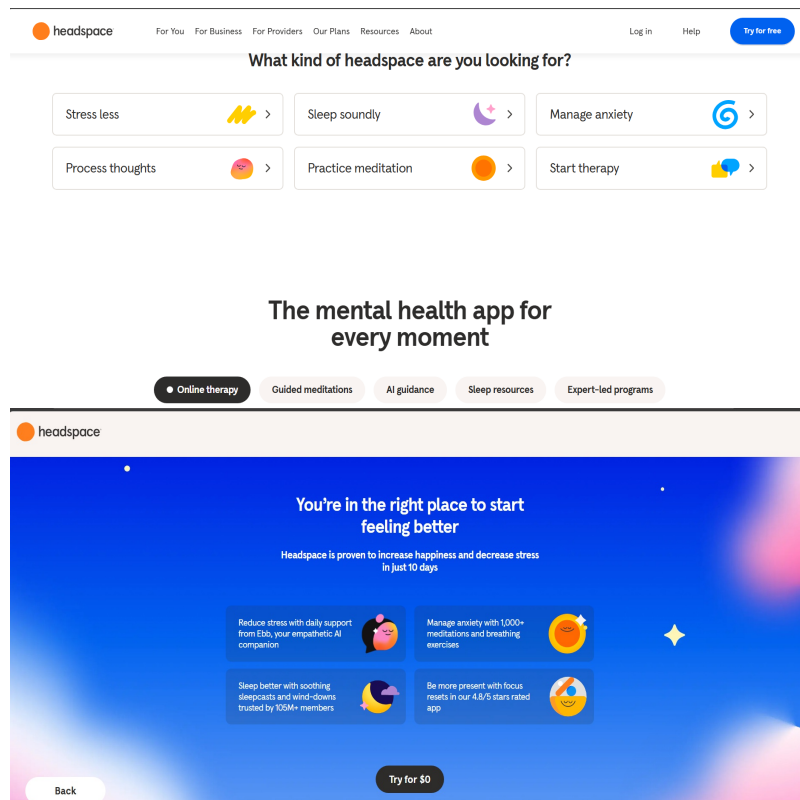
Calm

Calm je aplikacija za meditaciju, san i opuštanje koja nudi širok izbor sadržaja. Calm nudi meditacije, priče, glazbu, zvučne pejzaže i još mnogo toga, kako bi svoje korisnike odvela na novu razinu smirenosti. Za razliku od naše aplikacije, Calm je više generalna aplikacija za meditaciju i san, bez toliko detaljnog praćenja navika i mood trackinga. Također Calm nema implementirane izazove i streakove, pa ni integraciju s uređajima za spavanje, trenera i personalizirane preporuke i razlozima za te preporuke.



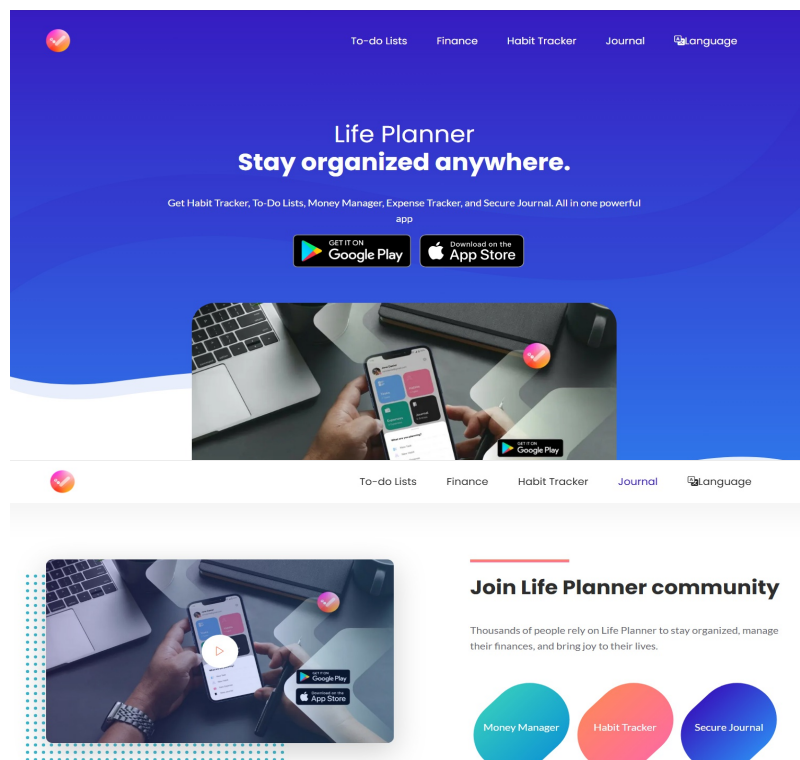
Headspace

Headspace je aplikacija prvenstveno orijentirana na vođene meditacije i mindfulness vježbe, s naglaskom na smanjenje stresa, bolji san i fokus. Headspace pruža alate za mindfulness za svakodnevni život, terapiju i psihijatriju. Ova aplikacija nije toliko fokusirana na vođenje dnevnika aktivnosti (daily check-in) te njegove personalizacije s "planom terapije".



Life Planner

Life Planner se za razliku od prethodne dvije aplikacije fokusira na praćenje navika kao pijenje vode, unos kofeina i slično. Aplikacija također omogućuje vođenje dnevnika. Life planner ne nudi mogućnost praćenja vođenih meditacija ili izvježbe mindfulnessa koliko je samo fokusirana na praćenje navika. Također ne pruža personalizirani sadržaj poput naše aplikacije.



Life Planner app users sharing

Naša aplikacija objedinjuje funkcionalnosti svih prethodno navedenih aplikacija te korisniku omogućuje kohezivno integrirano iskustvo. Korisnik unutar jedne aplikacije (Mindfulness) ima mogućnost praćenja meditacije i treninga te praćenja osobnih navika te vođenja dnevnika

Skup korisnika zainteresiran za ostvareno rješenje

Adolescenti, kao osobe koje se suočavaju s socijalnim izazovima, akademskim pritiskom i formiranjem osobnosti te traže diskretan i pristupačan način da dobiju podršku.

Roditelji i skrbnici, kao ljudi koji žele bolje navigirati obiteljski i profesionalni život, te podržati svoju djecu i partnere u mentalnom zdravlju.

Osobe u terapiji ili nakon terapije koji bi aplikaciju koriste kao dodatak terapiji te kao mjesto za zapisati svoje emocije.

Starije osobe koje trebaju podršku za svoju usamljenost, tugu ili anksioznost. Stručnjaci za mentalno zdravlje koji bi htjeli preporučiti alat za napredovanje klijentima te bi mogli pratiti napredak korisnika.

Svi ljudi zainteresirani za osobni razvoj, tj. tip ljudi koji ne pate, ali žele razviti bolju emocionalnu osviještenost te korisnici koji vole motivacijske programe i sl.

Mogućnost prilagodbe rješenja

Aplikacija je trenutačno na jednom jeziku. Aplikacija bi mogla podržavati više jezika što bi omogućilo korištenje u različitim dijelovima svijeta.

Aplikacija bi mogla biti pristupačna ljudima s invaliditetom poput sljepoće, tako da im se čita sadržaj ekrana.

Aplikacija bi mogla imati podesivu veličinu fonta kako bi bila pristupačnija seniorima.

Opseg projektnog zadatka

U sklopu projekta planirano je ostvariti sljedeće funkcionalnosti i komponente.

U skladu s dijagramom obrazaca uporabe, definirane su različite uloge korisnika, uključujući korisnike, trenere te administratore. Svaka od navedenih uloga ima različite ovlasti i pristup određenim funkcionalnostima u aplikaciji, čime se omogućava specifičan pristup podacima i mogućnostima koje aplikacija nudi za svaki tip korisnika.

Onboarding proces uključivat će kratak početni upitnik kojim korisnik opisuje svoju trenutnu razinu stresa, kvaitetu sna, iskustvo s meditacijom i osobne ciljeve. Na temelju prikupljenih informacija aplikacija će generirati personalizirani sadržaj.

Sustav preporuka koristit će podatke o korisničkim ciljevima, povijesti korištenja i drugo kako bi predložio najrelevantniji sadržaj. Baza podataka omogućit će brzi dohvat informacija za korisnike koji trebaju ažurirati podatke u stvarnom vremenu.

Kako bi osigurali sigurnost korisničkih podataka, pa tako i naših korisnika, autentifikacija i registracija korisnika provodit će se korištenjem sigurnosnih standarda poput OAuth 2.0. te tradicionalno, putem e-pošte i lozinke ili opcionalnom dvofaktorskom autentifikacijom. U slučaju registracije lozinkom, korisnik će biti obvezan promijeniti inicijalnu lozinku pri ponovnom prijavljivanju. Navedenim mjerama spriječit ćemo zlouporabu podataka i zaštititi aplikaciju od neovlaštenih pristupa.

Moguće nadogranje projektnog zadatka

Dodavanje dodatnih društvenih funkcionalnosti, poput međusobne komunikacije korisnika, komunikacije korisnika i trenera i slično. Proširenje personalizacije aplikacije, poput dodavanja odabira tamnog ili svijetlog načina rada aplikacije te prilagođavanje palete boja aplikacije po preferenci. Dodavanje dodatnih funkcionalnosti pretplatom na premium plan aplikacije. Za one koji ne žele napraviti korisnički račun omogućiti prijavu napraviti anonimno. Offline sadržaj, mogućnost preuzimanja meditacija, podcasta, članaka i sl.

U ovom poglavlju detaljno razraditi postavljene zahtjeve Funkcionalni zahtjevi: Opis funkcionalnosti aplikacije (npr., registracija korisnika, prijava slučajeva, pregled statusa pomoći). Nefunkcionalni zahtjevi: npr. Zahtjevi vezani uz performanse, sigurnost, skalabilnost i korisničko sučelje. Zahtjeve prikazati standardnim formatom dobro strukturirane tablice zahtjeva koja u SRS dokumentu uobičajeno uključuje stupce: ID zahtjeva, Opis, Prioritet, Izvor i Kriteriji prihvatanja (inačica ...). Primjer:

Funkcionalni zahtjevi

ID zahtjeva	Opis	Prioritet	Izvor	Kriteriji prihvatanja
F-001	Sustav omogućuje korisnicima registraciju i autentikaciju putem OAuth servisa ili e-mail/lozinka, uključujući obveznu promjenu inicijalne lozinke pri prvom pristupu.	Visok	Dokument zahtjeva	Korisnik se može uspješno registrirati te ako je registriran lozinkom pri prvom prijavljivanju mora promijeniti inicijalnu lozinku i pristupiti svom korisničkom računu.
F-002	Sustav nudi opciju aktivacije dvofaktorske autentikacije radi povećanja sigurnosti korisničkog pristupa.	Srednji	Dokument zahtjeva	Korisnik može aktivirati dvofaktorsku autentikaciju i koristi ju prilikom autentikacije na platformu.
F-003	Sustav omogućuje onboarding upitnik za nove korisnike na temelju kojeg se automatski generira personalizirani sedmodnevni plan aktivnosti	Visok	Dokument zahtjeva	Korisnik prolazi onboarding, upisuje relevantne podatke i dobiva plan aktivnosti prilagođen osobnim ciljevima.
F-004	Aplikacija korisniku generira svakodnevni izazov koji se sastoji od meditacije, vježbe te pitanja za praćenje raspoloženja i navika.	Srednji	Dokument zahtjeva	Korisniku se svakodnevno prikazuje novi izazov te se evidentira kontinuirani napredak (streak) na platformi.
F-005	Sustav omogućuje pregled i konzumaciju multimedijskog sadržaja (vodene meditacije, mindfulness vježbe, edukativni članci i podcasti).	Visok	Povratne informacije korisnika	Korisnik može pregledati te odabrati sadržaj za korištenje.
F-005.1	Sustav omogućuje filtriranu pretragu.	Srednje	Zahtjev dionika	Korisnik može pretraživati sadržaj te uključiti filtre kako bi sadržaj više odgovarao njegovim potrebama.
F-006	Korisnik ima mogućnost dnevnog unosa raspoloženja na skali 1–10, označavanja emocija te evidencije sna, stresa, fizičke aktivnosti, kofeina i alkohola te dodatnih bilješki.	Srednji	Povratne informacije korisnika	Korisnik može za odabrani dan unijeti i pregledati sve podatke vezane za mentalno stanje i navike u sučelju aplikacije.
F-007	Platforma je integrirana s vanjskim uređajima i uslugama za praćenje sna radi uvoza „sleep score” metrika i drugih relevantnih podataka.	Srednji	Dokument zahtjeva	Nakon povezivanja uređaja, relevantne metrike sna prikazuju se unutar korisničkog profila.
F-008	Sustav implementira algoritme preporuke koji predlažu sadržaj temeljen na ciljevima, navikama, doba dana, povijesti korištenja i trenutnom psihofizičkom stanju korisnika, uz jasno objašnjenje prijedloga.	Visok	Dokument zahtjeva	Korisniku se na početnoj stranici prikazuje personaliziran popis preporučenih sadržaja uz objašnjenje kriterija preporuke.
F-009	Platforma omogućuje korisnicima planiranje i zakazivanje praksi te sinkronizaciju termina s vanjskim kalendarima (Google Calendar).	Srednji	Dokument zahtjeva	Korisnik može dodati termine u aplikaciji te ih sinkronizirati i pregledati u povezanom vanjskom kalendaru.
F-010	Sustav podržava slanje obavijesti, automatiziranih podsjetnika (e-mail i push notifikacije) za planirane aktivnosti, kao i periodičnih motivacijskih poruka.	Srednji	Dokument zahtjeva	Korisniku se isporučuju obavijesti te podsjetnici točno prema zadanom rasporedu i korisničkim preferencijama.
F-011	Nakon završetka svake sesije, korisnici imaju mogućnost ocjenjivanja sadržaja u rasponu 1–5 zvjezdica te ostavljanja komentara, uz javno prikazivanje prosječnih ocjena.	Srednji	Povratne informacije korisnika	Ocjene i komentari su javno vidljivi, prosječna ocjena prikazuje se uz sadržaj, a administrator ima mogućnost moderiranja neprikladnih recenzija.
F-012	Sustav omogućuje sučelje za administratore/moderatore.	Srednji	Zahtjev dionika	Administrator ima mogućnost upravljanja svim resursima unutar platforme, provodi moderaciju, ima pristup blokiranju računa, može izraditi sigurnosnu kopiju sustava i analizirati audit logove bez prekida usluge.
F-013	Sustav korisniku omogućuje prikaz statistike napretka.	Srednji	Dokument zahtjeva	Korisnik može doći do statističkih prikaza vlastitih navika, uključujući broj uzastopnih dana korištenja, dovršene izazove, trendove raspoloženja i tjedne sažetke aktivnosti.

ID zahtjeva	Opis	Prioritet	Izvor	Kriteriji prihvatanja
F-014	Sustav omogućuje sučelje za trenera/terapeuta.	Visok	Zahtjev dionika	Trener/terapeut može objavljivati programe i planove te pratiti napredak korisnika ako korisnik to dopusti.
F-015	Sustav omogućuje korisniku pregled svojeg profila.	Srednji	Zahtjev dionika	Korisnik na stranici profila može pregledati svoje podatke i osvojene značke.
F-015.1	Sustav omogućuje korisniku uređivanje svojeg profila.	Srednji	Zahtjev dionika	Korisnik na stranici profila može promijeniti osobne podatke.

Ostali zahtjevi

- NF - ne-funkcijski zahtjevi
- Zahtjevi održavanja/performansi i dostupnosti - 1.1
- Zahtjevi dokumentacije - 1.2
- Zahtjevi sigurnosti - 1.3
- Zahtjevi skalabilnosti - 1.4
- Zahtjevi upotrebljivosti - 1.5

ID zahtjeva	Opis	Prioritet
NF - 1.1.0	Sustav treba biti oblikovan tako da omogućuje jednostavno održavanje	Visok
NF - 1.1.1	Vrijeme učitavanja stranica i odgovora sustava na zahtjeve korisnika mora biti manje od 2 sekunde	Visok
NF - 1.1.2	Sustav treba podržavati velik broj korisnika bez pada performasi	Visok
NF - 1.1.3	Sustav mora biti dostupan najmanje 99% vremena	Srednji
NF - 1.3.0	Osjetljivi podaci korisnika moraju biti zaštićeni pristupnim kontrolama i dostupni samo ovlaštenim korisnicima	Visok
NF - 1.4.1	Sustav treba biti oblikovan tako da omogućuje jednostavno proširivanje	Visok
NF - 1.5.0	Aplikacija treba biti jednostavna i intuitivna za korištenje	Visok
NF - 1.5.1	Sustav mora biti kompatibilan s popularnim web preglednicima	Visok
NF - 1.5.2	Aplikacija mora imati responzivan dizajn i biti prilagođena za mobilne uređaje	Srednji

Dionici

- Korisnici sustava
 - korisnik
 - trener/terapeut
- Neprijavljeni korisnici
- Administratori sustava
- Razvojni tim
- Naručitelji sustava

Aktori i njihovi funkcionalni zahtjevi:

Korisnici sustava (A-1)

- Funkcije: F-001, F-002, F-003, F-004, F-005, F-005.1, F-006, F-008, F-009, F-010, F-011, F-013, F-015, F-015.1

Treneri/terapeuti (A-2)

- Funkcije: F-001, F-002, F-003, F-004, F-005, F-005.1, F-006, F-008, F-009, F-010, F-011, F-013, F-014, F-015, F-015.1

Administratori (A-3)

- Funkcije: F-001, F-002, F-005, F-005.1, F-010, F-012, F-014

Neprijavljeni korisnici (A-4)

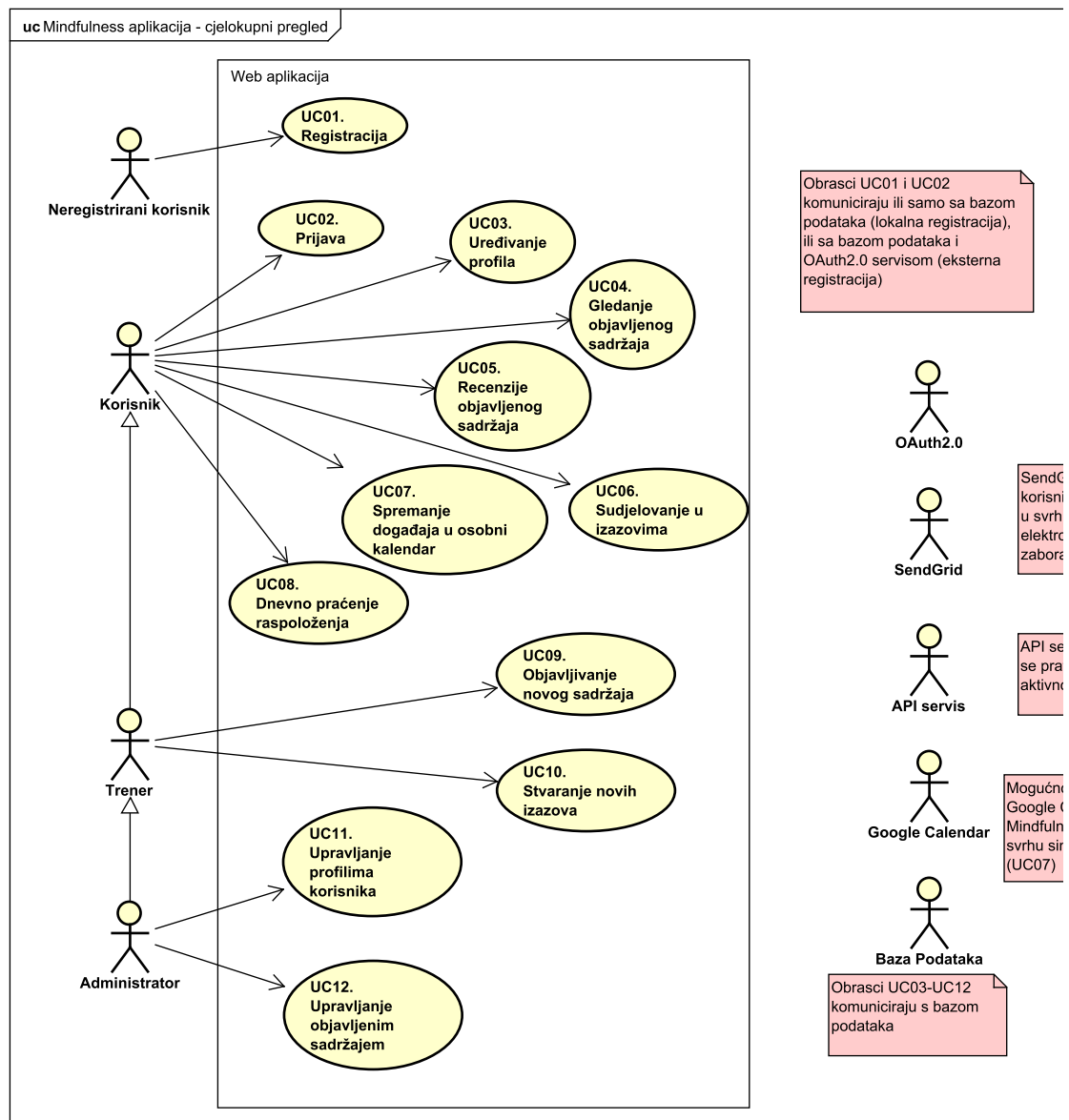
- Funkcije: F-001

Obrasci uporabe

Dijagrami obrazaca uporabe

Prkazati odnos aktora i obrazaca uporabe odgovarajućim UML dijagramom. Nije nužno nacrtati sve na jednom dijagramu. Modelirati po razinama apstrakcije i skupovima srodnih funkcionalnosti.

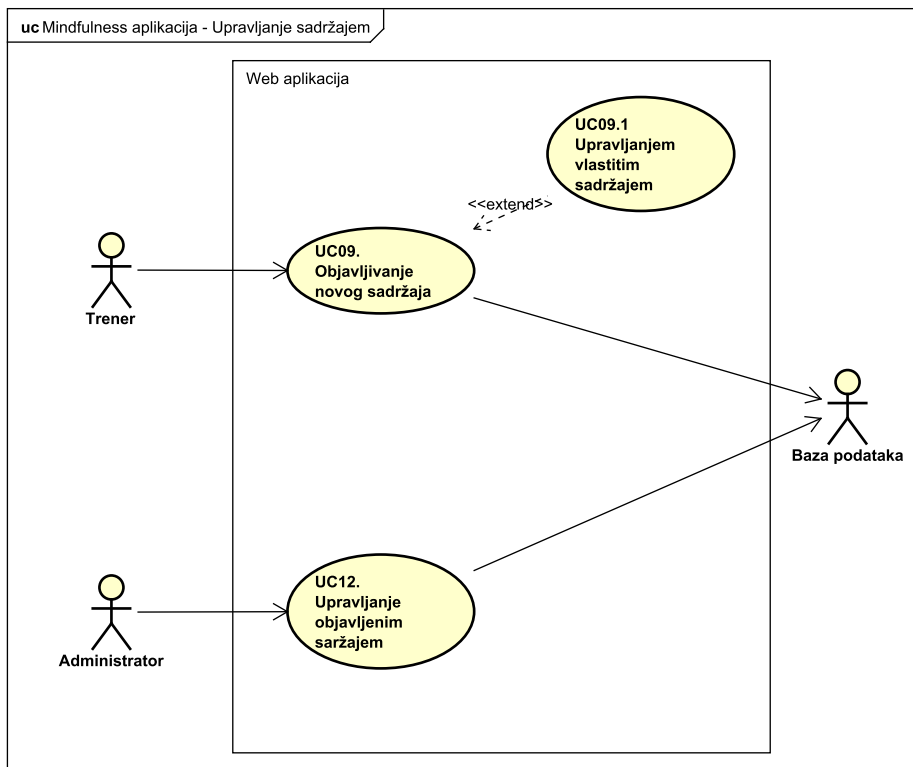
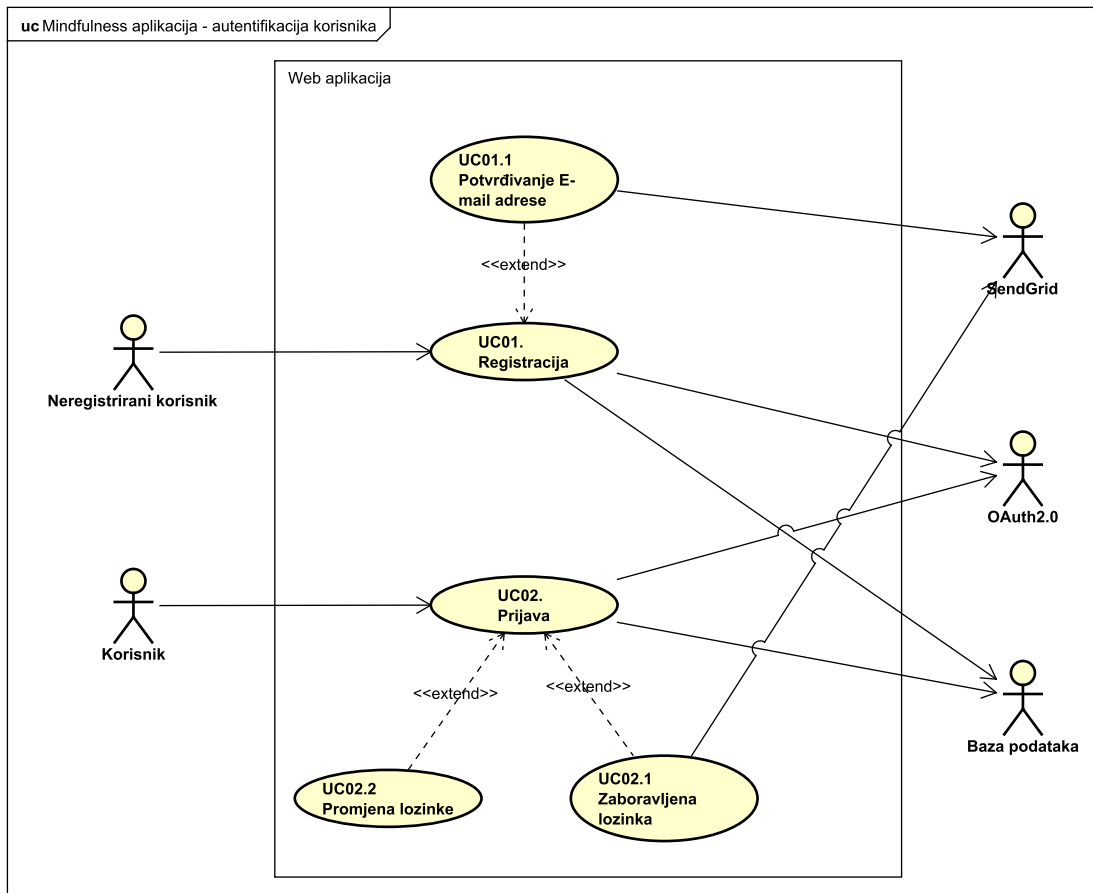
1. Visokorazinski dijagram obrazaca uporabe cijelog sustava

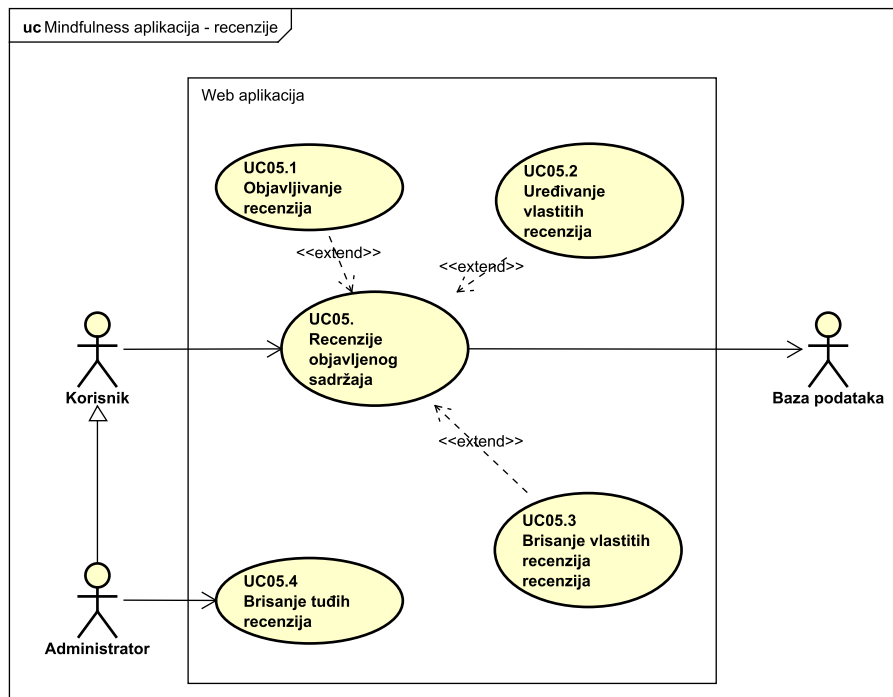
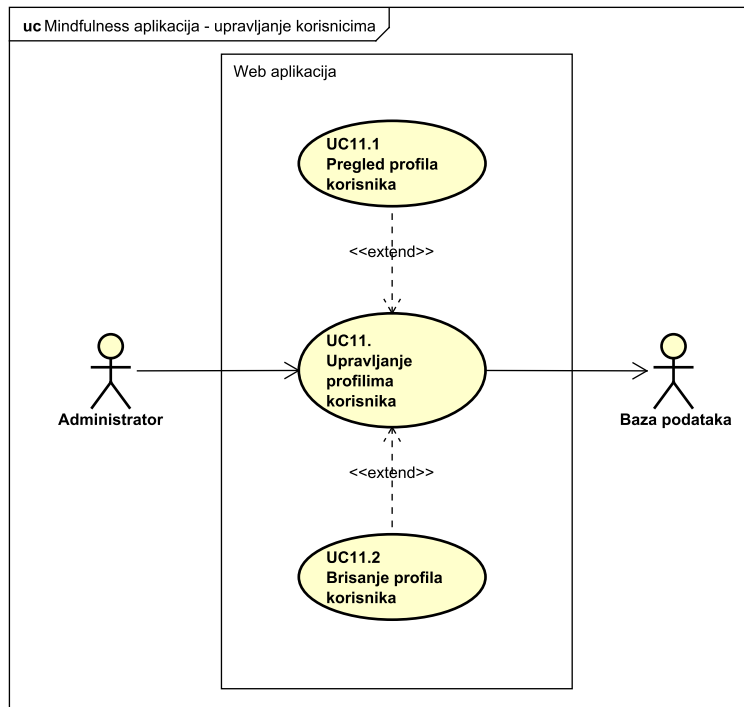


| Slika sustava - osnova za razumijevanje glavnih ciljeva projekta.

2. dijagram obrazaca uporabe za ključne značajke

| detaljniji dijagrami koji se odnose na specifične značajke, poput procesa prijave korisnika, upravljanja podacima, ... obrade transakcija.



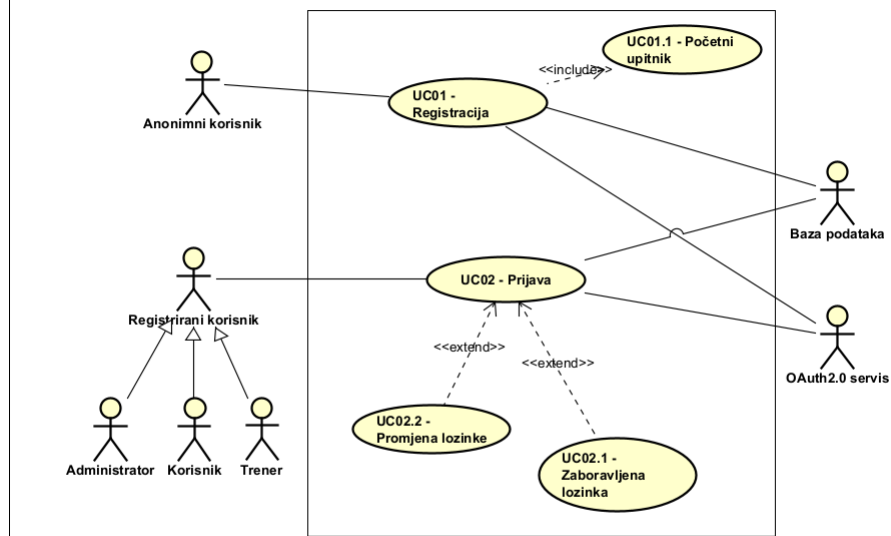


3. dijagram obrazaca uporabe za korisničke uloge

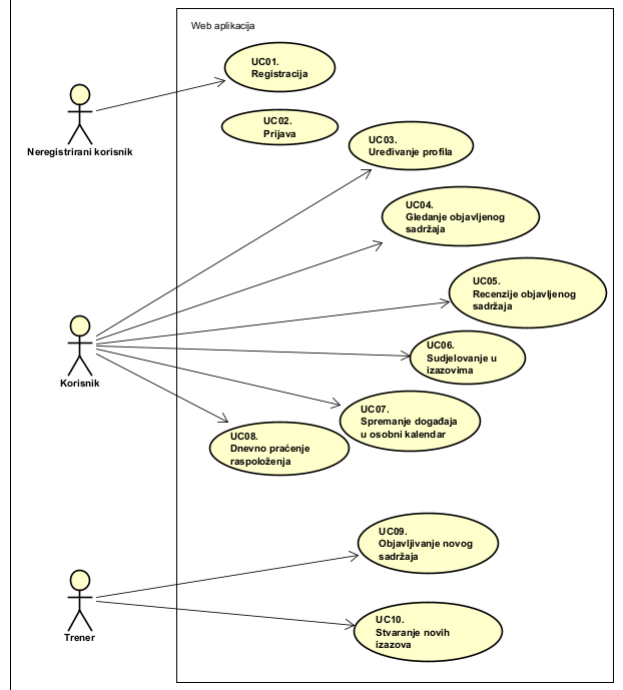
| dijagram za korisničke uloge

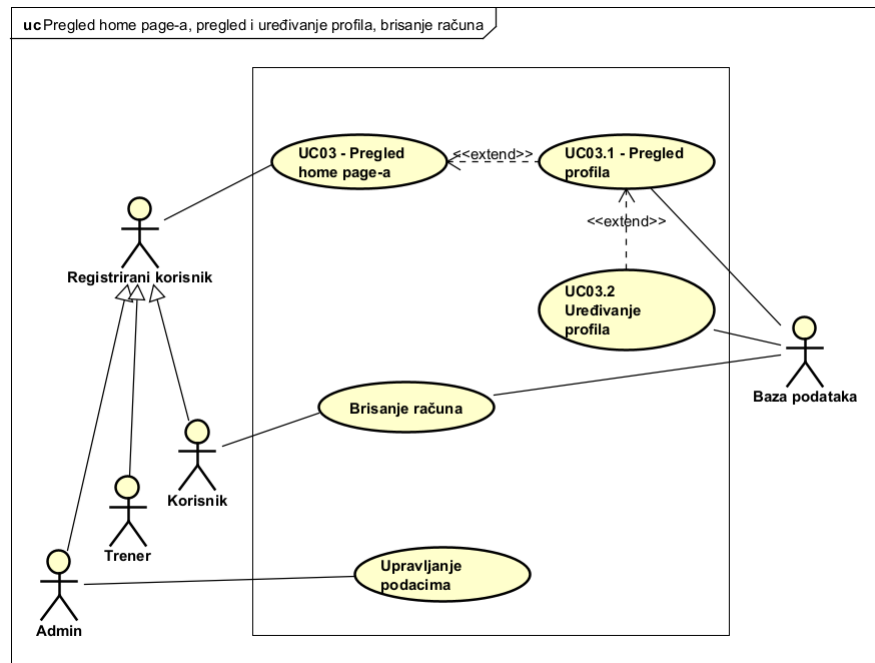
| Identificira glavne korisničke aktore i njihove interakcije sa sustavom.

uc Registracija, prijava, zaboravljena lozinka



uc Mindfulness aplikacija - korisničke uloge

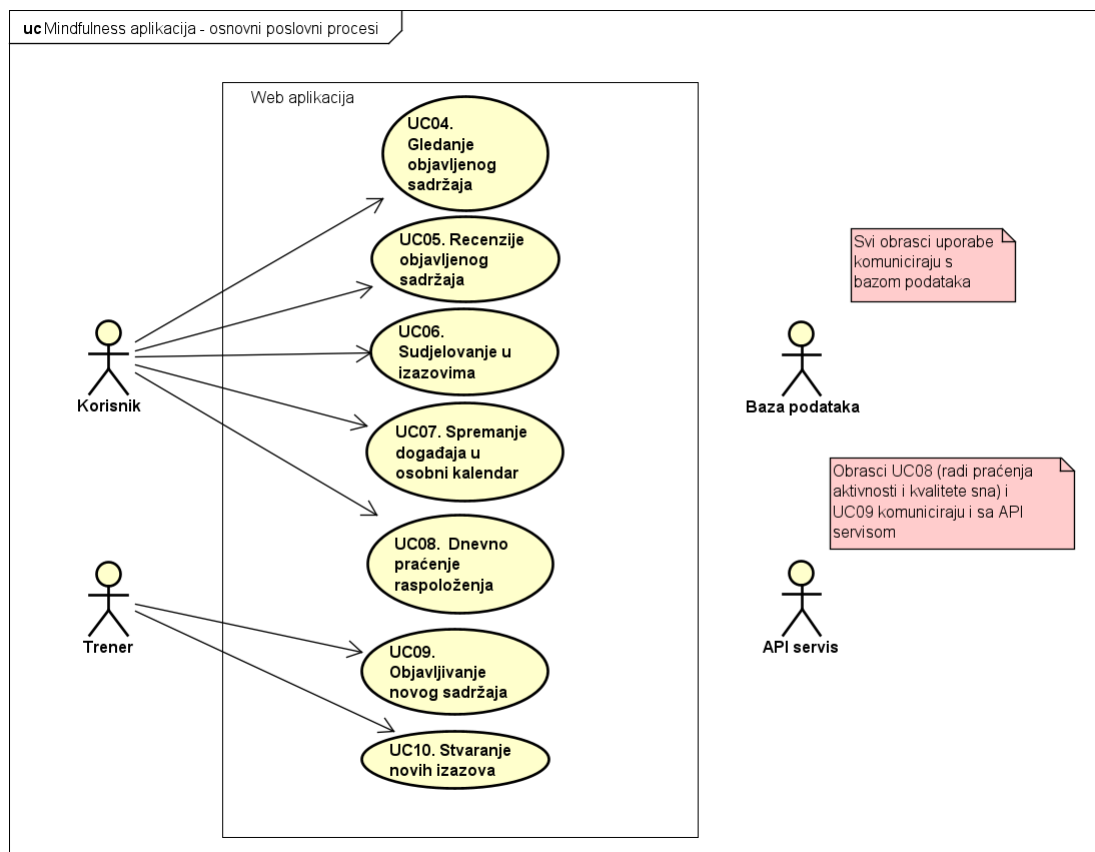




| Definira različiti vrsta korisnika i funkcionalnosti koje su im potrebne te sigurava bolje razumjevanje prioritete i specifične potrebe različitih grupa korisnika.

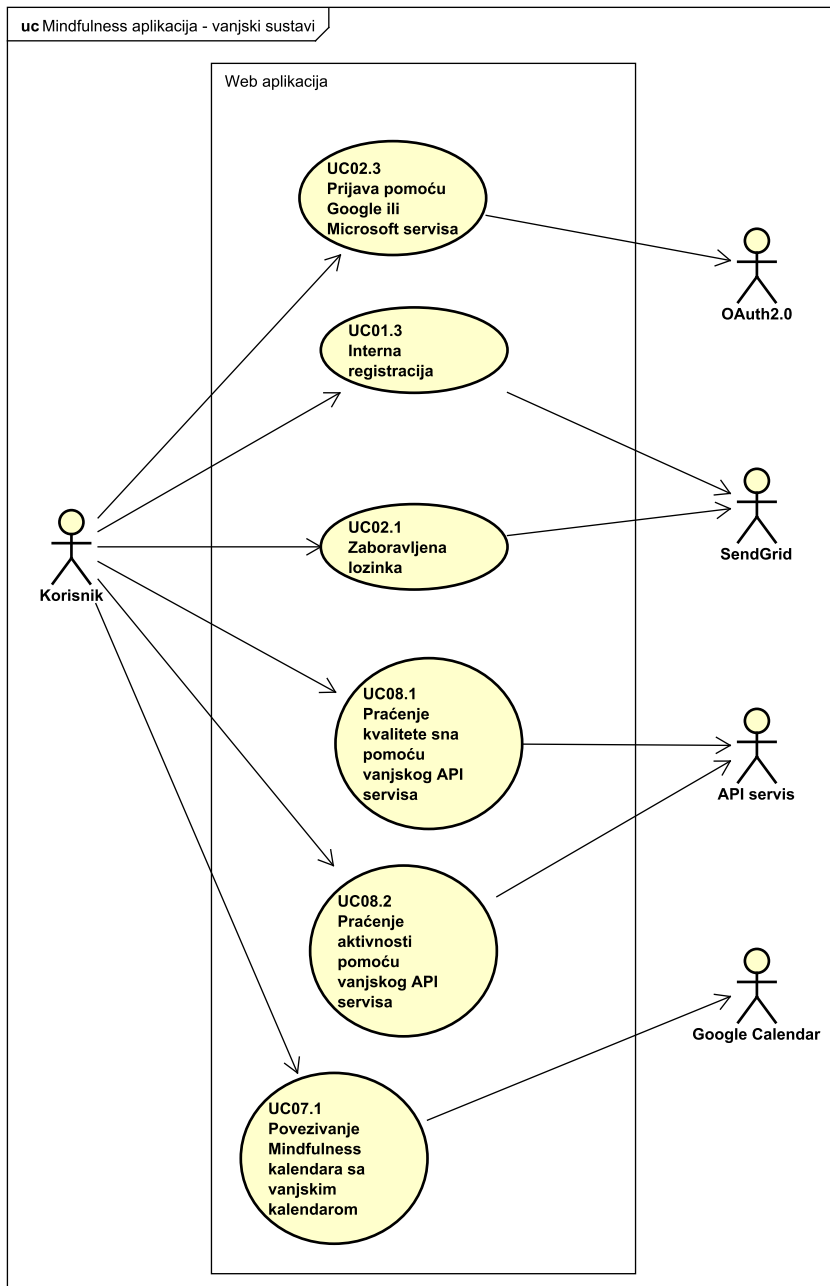
4. dijagram obrazaca uporabe za osnovne poslovne procese

| prikazuje kako sustav podržava osnovne poslovne procese organizacije.



5. dijagram obrazaca uporabe za kritične sustave i integracije

| interakcije sustava s vanjskim sustavima ili integracijama



Opis obrazaca uporabe

UC01 - Registracija (s e-mail adresom)

- Glavni sudionik: Neprijavljeni korisnik
- Cilj: Registrirati se u sustav, stvoriti korisniku profil.
- Sudionici: baza podataka, mailing servis, OAuth servis
- Preduvjet: korisnik nema profil
- **Opis osnovnog tijeka:**
 1. Korisnik pritisne tipku registracija
 2. Prikazuje mu se stranica za registraciju gdje ima opciju registrirati se putem OAuth servisa ili e-mailom i lozinkom
 3. Korisnik upisuje podatke kako bi se registrirao te stisne na gumb registriraj se:
 4. Korisniku se prikaže poruka da mora potvrditi svoju e-mail adresu, te mu na e-mail dolazi link za potvrđivanje
 5. Korisnik potvrđuje e-mail adresu te odlazi na prijavu
 6. Korisnik se prijavljuje te ga vodi na početni upitnik
 7. Korisnik predaje odgovore upitnika
 8. Zahtjeva se inicijalna promjena lozinke jer se prijavio e-mail adresom i lozinkom
 9. Korisnik mijenja lozinku te dobije pristup aplikaciji
- **Opis mogućih odstupanja:**
 1. E-mail adresa je već registrirana - sustav obavještava korisnika
 2. Lozinka ne zadovoljava sigurnosne kriterije - sustav obavještava korisnika

3. Korisniku ne stiže e-mail sa verifikacijskim linkom - korisnik može tražiti da mu se ponovno pošalje
4. Korisnik pri promjeni lozinke upisuje krivu staru lozinku - sustav obavještava korisnika

UC01.1 - Početni upitnik

- Glavni sudionik: Korisnik
- Cilj: Skupljanje informacija za generaciju sedmodnevnog plana aktivnosti
- Sudionici: baza podataka
- Preduvjet: korisnik nije prije rješavao početni upitnik
- **Opis osnovnog tijeka:**
 1. Nakon registracije sustav korisnika vodi na početni upitnik
 2. Korisnik odgovara na pitanja
 3. Korisnik predaje odgovore
 4. Sustav sprema odgovore te pomoću njih generira personalizirani sedmodnevni plan aktivnosti
- **Opis mogućih odstupanja:**
 1. Korisnik želi preskočiti pitanje - sustav ga obavještava da ne može jer je pitanje obavezno

UC02 - Prijava (s e-mail adresom)

- Glavni sudionik: Neprijavljeni korisnik
- Cilj: Prijaviti korisnika na njegov profil
- Sudionici: baza podataka, OAuth servis
- Preduvjet: korisnik postoji u bazi podataka - registriran je
- **Opis osnovnog tijeka:**
 1. Korisnik pritisne tipku prijava
 2. Prikazuje mu se stranica za prijavu gdje ima opciju prijaviti se putem OAuth servisa ili e-mail adresom i lozinkom
 3. Korisnik upisuje e-mail adresu i lozinku koji su spremjeni u bazu podataka te stisne na gumb prijavi se:
 4. korisnik dobije pristup aplikaciji i svojem profilu
- **Opis mogućih odstupanja**
 1. Korisnik upisuje krivu e-mail adresu - sustav obavještava korisnika
 2. Korisnik upisuje krivu lozinku - sustav obavještava korisnika

UC02.1 - Zaboravljena lozinka

- Glavni sudionik: Neprijavljeni korisnik
- Cilj: Omogućiti korisniku promjenu lozinke iako nije prijavljen
- Sudionici: baza podataka, mailing servis
- Preduvjet: korisnik ima napravljen profil i nije prijavljen
- **Opis osnovnog tijeka:**
 1. Korisnik na sučelju za prijavu stisne na link za zaboravljenu lozinku
 2. Otvara mu se sučelje gdje upisuje e-mail adresu svojeg računa na koju će mu se poslati poveznica za ponovno postavljanje lozinke
 3. Mailing servis šalje e-mail sa poveznicom
 4. Korisnik otvara link i upisuje svoju novu lozinku
 5. Preusmjerava se na prijavu
- **Opis mogućih odstupanja:**
 1. Korisnik ne dobiva e-mail sa poveznicom - može zatražiti ponovno slanje

UC02.2 - Aktivacija dvofaktorske autentikacije

- Glavni sudionik: Prijavljeni korisnik
- Cilj: Omogućiti dodatnu sigurnost pri prijavi
- Sudionici: baza podataka, mailing servis
- Preduvjet: korisnik ima napravljen profil i prijavljen je
- **Opis osnovnog tijeka:**
 1. Korisnik u postavkama profila odabire opciju aktivacije dvofaktorske autentikacije
 2. Korisnik upisuje e-mail adresu na koju želi primiti verifikacijski kod
 3. Korisniku na e-mail adresu dolazi potvrda kako je dvofaktorska autentikacija aktivirana.
 4. Korisnika se odjavljuje te se mora ponovno prijaviti i koristiti dvofaktorsku autentikaciju.
 5. Korisnik se prijavljuje, dobiva verifikacijski kod na e-mail adresu te ga upisuje u sustav
 6. Korisnik ponovno dobiva pristup profilu
- **Opis mogućih odstupanja:**

1. Korisniku ne dolazi e-mail sa verifikacijskim kodom - korisnik može zatražiti ponovno slanje

UC02.3 Prijava pomoću Google servisa

- Glavni sudionik: Neprijavljeni korisnik
- Cilj: Prijaviti korisnika na njegov profil
- Sudionici: baza podataka, OAuth servis
- Preduvjet: korisnik nije prijavljen
- **Opis osnovnog tijeka:**
 1. Korisnik klikne tipku prijava
 2. Prikazuje mu se stranica za prijavu gdje ima opciju prijaviti se putem OAuth servisa ili e-mail adresom i lozinkom
 3. Korisnik stisne na OAuth gumb (Google):
 4. Preusmjerava ga se na stranicu OAuth servisa
 5. Korisnika se prijavljuje u sustav te dobije pristup aplikaciji i svojem profilu
- **Opis mogućih odstupanja:**
 1. Korisnik nije registriran - korisnika se automatski registrira i šalje na početni upitnik
 2. OAuth nije uspio autenticirati korisnika - sustav obavještava korisnika
 3. Korisnik je registriran sa e-mail adresom i lozinkom - korisnik se prepoznaje te ga se prijavljuje u sustav

UC03 - Pregled home page-a

- Glavni sudionik: Prijavljeni korisnik
- Cilj: Promjena osobnih podataka i preferenci
- Sudionici: baza podataka
- Preduvjet: korisnik je prijavljen
- **Opis osnovnog tijeka:**
 1. Korisnik pristupa početnoj stranici aplikacije
 2. Početna stranica omogućuje svim korisnicima pregled vlastitog home page-a

UC03.1 - Pregled profila profila

- Glavni sudionik: Prijavljeni korisnik
- Cilj: Promjena osobnih podataka i preferenci
- Sudionici: baza podataka
- Preduvjet: korisnik je prijavljen
- **Opis osnovnog tijeka:**
 1. Korisnik na stranici profila stisne gumb za postavke profila
 2. Korisniku se prikazuju detalji profila

UC03.2 - Uređivanje profila

- Glavni sudionik: Prijavljeni korisnik
- Cilj: Promjena osobnih podataka i preferenci
- Sudionici: baza podataka
- Preduvjet: korisnik je prijavljen
- **Opis osnovnog tijeka:**
 1. Korisnik na stranici profila stisne gumb za postavke profila
 2. Korisnik bira stavku koju želi promijeniti, te je mijenja
 3. Promjena se sprema u bazu podataka
 4. Sustav pokazuje potvrdu

UC03.3 Uređivanje e-mail adrese

- Glavni sudionik: Prijavljeni korisnik
- Cilj: Promjena e-mail adrese profila
- Sudionici: baza podataka, mailing servis
- Preduvjet: korisnik je prijavljen
- **Opis osnovnog tijeka:**
 1. Korisnik na sučelju postavki profila odabire promjenu e-mail adrese
 2. Otvara se sučelje za promjenu
 3. Korisnik upisuje novu e-mail adresu
 4. Sustav zahtjeva potvrdu nove adrese te šalje e-mail za potvrdu
 5. Korisnik potvrđuje e-mail adresu
 6. Sustav pokazuje potvrdu
- **Opis mogućih odstupanja:**

1. Korisnik ne dobiva e-mail za potvrdu - može zatražiti ponovno slanje

UC04 - Gledanje objavljenog sadržaja

- Glavni sudionik: Prijavljeni korisnik
- Cilj: Pregled objavljenih sadržaja (video, meditacija, članak...)
- Sudionici: baza podataka
- Preduvjet: korisnik je prijavljen
- **Opis osnovnog tijeka:**
 1. Korisnik je na dashboard stranici:
 - preporučuje mu se personalizirani sadržaj
 - korisnik stisne na sadržaj koji želi pregledati
 - Otvara mu se sadržaj
 2. Korisnik je na stranici sadržaja:
 - Sustav prikazuje sav sadržaj te kategorije
 - Korisnik primjenjuje filtere te odabire što želi pregledati
 - Otvara mu se sadržaj
 3. Korisnik ima mogućnost ostavljanja recenzije
 4. Zatvara sadržaj na gumb za zatvaranje

UC05.1 - Objavljivanje recenzija

- Glavni sudionik: Prijavljeni korisnik
- Cilj: Ostavljanje recenzije sadržaja
- Sudionici: baza podataka
- Preduvjet: korisnik je prijavljen
- **Opis osnovnog tijeka:**
 1. Korisniku se tijekom i nakon pregleda sadržaja nudi da ostavi recenziju i komentar
 2. Korisnik bira ocjenu koju želi ostaviti te piše komentar
 3. Recenzija se sprema u bazu podataka za kasnije pregledavanje
- **Opis mogućih odstupanja:**
 1. Korisnik ne želi ostaviti komentar - ocjena se idalje sprema

UC05.2 - Uređivanje vlastitih recenzija

- Glavni sudionik: Prijavljeni korisnik
- Cilj: Promjena vlastite recenzije sadržaja
- Sudionici: baza podataka
- Preduvjet: korisnik je prijavljen, ostavio je recenziju sadržaju
- **Opis osnovnog tijeka:**
 1. Korisniku se tijekom i nakon pregleda sadržaja prikazuje njegova recenzija
 2. Korisnik klikne na recenziju i mijenja je
 3. Korisnik potvrđuje da želi promijeniti recenziju
 4. Recenzija se sprema u bazu podataka za kasnije pregledavanje

UC05.3 - Brisanje vlastitih recenzija

- Glavni sudionik: Prijavljeni korisnik
- Cilj: Brisanje vlastite recenzije sadržaja
- Sudionici: baza podataka
- Preduvjet: korisnik je prijavljen, ostavio je recenziju sadržaju
- **Opis osnovnog tijeka:**
 1. Korisniku se tijekom i nakon pregleda sadržaja prikazuje njegova recenzija
 2. Korisnik klikne na recenziju i briše je
 3. Korisnik potvrđuje da želi izbrisati recenziju
 4. Recenzija se briše iz bazu podataka

UC05.4 - Brisanje tuđih recenzija

- Glavni sudionik: Administrator
- Cilj: Brisanje korisničkih recenzija ako nisu primjerene
- Sudionici: baza podataka
- Preduvjet: korisnik je ostavio neprimjerenu recenziju
- **Opis osnovnog tijeka:**
 1. Administratoru se u pregledu sadržaja prikazuju recenzije korisnika
 2. Administrator klikne na recenziju te dobije opciju za brisanje

3. Klikne na brisanje
4. Recenzija se briše iz baze podataka

UC06 - Sudjelovanje u izazovima

- Glavni sudionik: Korisnik
- Cilj: Sudjelovanje u nizu aktivnosti uključenih u izazov
- Sudionici: baza podataka
- Preduvjet: korisnik je prijavljen, sustav je generirao dnevni izazov ili povukao neki od postojećih
- **Opis osnovnog tijeka:**
 1. Korisnik na dashboard-u klikne na gumb za dnevni izazov
 2. Otvara se sučelje izazova gdje su prikazani zadaci koje korisnik mora odraditi
 3. Korisnik započinje izazov
 4. Sustav prati napredak tijekom sesije
 5. Korisnik završava izazov
 6. Sustav bilježi kraj i ažurira streak
- **Opis mogućih odstupanja:**
 1. Korisnik se izlogira iz aplikacije tijekom izvršavanja izazova - sustav sprema napredak te korisnik nastavlja tamo gdje je stao ako nastavi tijekom istog dana
 2. Korisnik ne završi izazov u istom danu - streak se prekida

UC07.1 - Spremanje događaja u kalendar sa stranice kalendara

- Glavni sudionik: Korisnik
- Cilj: Praćenje rasporeda aktivnosti spremanjem događaja u kalendar
- Sudionici: baza podataka
- Preduvjet: korisnik je prijavljen
- **Opis osnovnog tijeka:**
 1. Korisnik pristupa stranici kalendara
 2. Korisnik bira datum i vrijeme za sesiju
 3. Korisnik bira tip aktivnosti
 4. Korisnik stisne spremi
 5. Sustav kreira događaj u internom kalendaru

UC07.2 Brisanje događaja iz kalendara

- Glavni sudionik: Korisnik
- Cilj: Brisanje događaja ako vrijeme više na odgovara
- Sudionici: baza podataka
- Preduvjet: korisnik je prijavljen, događaj postoji
- **Opis osnovnog tijeka:**
 1. Korisnik pristupa stranici kalendara
 2. Korisnik klikne na datum za koji želi izbrisati događaj
 3. Otvara se lista događaja za taj datum
 4. Korisnik klikne na događajte mu se prikazuju detalji događaja
 5. Korisnik klikne na gumb za brisanje
 6. Sustav traži potvrdu za brisanje
 7. Korisnik daje potvrdu
 8. Sustav briše događaj iz baze podataka
 9. Sustav šalje potvrdu o brisanju

UC07.3 - Spremanje događaja u kalendar kroz aktivnost

- Glavni sudionik: Korisnik
- Cilj: Praćenje rasporeda aktivnosti spremanjem događaja u kalendar
- Sudionici: baza podataka
- Preduvjet: korisnik je prijavljen
- **Opis osnovnog tijeka:**
 1. Korisnik otvara neku aktivnost (video, članak, meditacija...)
 2. Prikazuje se opcija za dodavanje u kalendar
 3. Korisnik klikne na gumb dodaj u kalendar
 4. Bira datum i vrijeme te klikne spremi događaj
 5. Sustav kreira događaj u internom kalendaru

UC07.4 - Sinkronizacija kalendara s Google Calendarom

- Glavni sudionik: Korisnik
- Cilj: Praćenje rasporeda aktivnosti sinkronizacijom kalendara s vanjskim kalendarom

- Sudionici: baza podataka, Google Calendar API
- Preduvjet: korisnik je prijavljen, korisnik ima Google račun
- **Opis osnovnog tijeka:**
 1. Korisnik ulazi na stranicu kalendara
 2. Sustav nudi opciju Sinkroniziraj sa Google Calendarom
 3. Korisnik klikne na tu opciju
 4. Sustav autorizira pristup Google Calendar API-ju
 5. Sustav kreira događaj u Google Calendaru
 6. Sustav potvrđuje uspješnu sinkronizaciju
- **Opis mogućih dostupanja:**
 1. Sinkronizacija nije uspjela - sustav obavještava korisnika

UC08 - Dnevno praćenje raspoloženja

- Glavni sudionik: Korisnik
- Cilj: Praćenje navika korisnika kroz statistiku
- Sudionici: baza podataka
- Preduvjet: korisnik je prijavljen
- **Opis osnovnog tijeka:**
 1. Sustav na dashboard-u i na stranici statistike ima opciju "Dnevni Check-in"
 2. Korisnik klikne na opciju te mu se otvara sučelje za unos raspoloženja i emocija
 3. Korisnik ispunjava anketu
 4. Sustav prati napredak
 5. Korisnik klikne spremi te sustav sprema odgovore u bazu podataka
- **Opis mogućih odstupanja:**
 1. Korisnik preskoči neko pitanje - sustav sprema ostale odgovore

UC08.1 - Praćenje kvalitete sna pomoću vanjskog API servisa

- Glavni sudionik: Korisnik
- Cilj: Detaljno praćenje kvalitete sna korisnika pomoću uređaja za praćenje sna
- Sudionici: baza podataka, API servis
- Preduvjet: korisnik je prijavljen, korisnik ima uređaj za praćenje sna
- **Opis osnovnog tijeka:**
 1. Sustav na stranici statistike nudi opciju povezivanja vansjkih uređaja za praćenje sna
 2. Korisnik klikne na tu opciju
 3. Korisnik bira koji uređaj želi spojiti
 4. Sustav se preko API servisa spaja sa uređajem i sinkronizira podatke
 5. Korisniku se otvara novi dio sučelja za detaljnije praćenje sna

UC08.2 - Praćenje aktivnosti pomoću vanjskog API servisa

- Glavni sudionik: Korisnik
- Cilj: Praćenje aktivnosti korisnika kroz druge aplikacije i uređaje
- Sudionici: baza podataka, API servis
- Preduvjet: korisnik je prijavljen
- **Opis osnovnog tijeka:**
 1. Sustav na stranici statistike nudi opciju sinkronizacije aktivnosti sa vanskim uređajima i aplikacijama
 2. Korisnik klikne na tu opciju
 3. Korisnik bira odakle želi sinkronizirati aktivnost
 4. Sustav se preko API servisa spaja i sinkronizira podatke
 5. Korisniku se prikazuju podaci o aktivnosti

UC08.3 - Pregled statistika iz dnevnog praćenja raspoloženja

- Glavni sudionik: Korisnik
- Cilj: Pregled napretka korisnika kroz statistike
- Sudionici: baza podataka
- Preduvjet: korisnik je prijavljen
- **Opis osnovnog tijeka:**
 1. Sustav nudi stranicu statistike
 2. Korisnik klikne na ikonu stranice statistike
 3. Sustav prikazuje korisniku sve njegove statistike
 4. Korisnik klikne na određenu statistiku

5. Sustav prikazuje podatke u obliku grafa po danima, tjednima, mjesecima ili godinama

- **Opis mogućih odstupanja:**

1. Korisnik u nekom check-in-u nije odgovorio na neka pitanja - sustav prikazuje statistike bez tih podataka

UC09 - Objavljivanje novog sadržaja

- Glavni sudionik: Trener/terapeut (korisnik), Administrator
- Cilj: Dodavanje novog sadržaja u aplikaciju
- Sudionici: baza podataka, API servis
- Preduvjet: korisnik je prijavljen

- **Opis osnovnog tijeka:**

1. Sustav u sučelju za trenere ima opciju dodavanja novog sadržaja
2. Trener klikne na stranicu za dodavanje novog sadržaja
3. Trener odabire koju vrstu sadržaja dodaje
4. Trener prilaže datoteku
5. Klikne objavi
6. Sustav sprema sadržaj i dodaje ga u bazu podataka
7. Sustav šalje potvrdu o objavljivanju

UC09.1 - Upravljanje vlastitim sadržajem

- Glavni sudionik: Trener/terapeut (korisnik), Administrator
- Cilj: Uređivanje objavljenog sadržaja nakon objavljivanja, poboljšavanje i slično
- Sudionici: baza podataka
- Preduvjet: korisnik je prijavljen

- **Opis osnovnog tijeka:**

1. Sustav u sučelju za trenere nudi opciju pregleda objavljenog sadržaja
2. Trener klikne na sadržaj
3. Sustav nudi opciju uređivanja
4. Trener klikne na uređivanje sadržaja
5. Trener unosi željene promjene i stisne spremi
6. Sustav traži potvrdu
7. Trener potvrđuje
8. Sustav sprema promjene i šalje potvrdu o promjeni

- **Opis mogućih odstupanja:**

1. Administrator izbrisao sadržaj - trener neće moći pristupiti sadržaju

UC10 - Stvaranje novih izazova

- Glavni sudionik: Trener/terapeut (korisnik), Administrator
- Cilj: Stvaranje novih izazova koji će se korisnicima pojaviti u dnevnom izazovu
- Sudionici: baza podataka
- Preduvjet: korisnik je prijavljen

- **Opis osnovnog tijeka:**

1. Sustav u sučelju za trenere nudi opciju stvaranja izazova
2. Trener klikne na opciju
3. Trner bira težinu izazova, sadržaje te na što se izazov fokusira
4. Trener klinke stvori
5. Sustav sprema izazov za kasniju upotrebu
6. Sustav šalje potvrdu o stvaranju

UC11.1 - Pregled profila korisnika

- Glavni sudionik: Administrator
- Cilj: Pregled statistika, administrator je ujedno i trener
- Sudionici: baza podataka
- Preduvjet: korisnik postoji
- **Opis osnovnog tijeka:**
 1. Sučelje za administratora nudi stranicu za pregled profila korisnika
 2. Administrator bira profil koji želi pogledati
 3. Prikazuju mu se podaci, statistike i recenzije korisnika

UC11.2 - Brisanje profila korisnika

- Glavni sudionik: Administrator
- Cilj: Brisanje neprimjerenih korisnika
- Sudionici: baza podataka

- Preduvjet: korisnik postoji
- **Opis osnovnog tijeka:**
 1. Sustav u sučelju za pregled profila korisnika nudi opcija brisanja profila korisnika
 2. Administrator pronalazi korisnika čiji profil želi izbrisati te klikne na njega
 3. Klikne na gumb izbriši profil
 4. Sustav traži potvrdu za brisanje
 5. Administrator daje potvrdu
 6. Sustav briše profil i sve vezano za profil iz baze podataka
 7. Sustav šalje potvrdu o brisanju

UC12 - Upravljanje objavljenim sadržajem

- Glavni sudionik: Administrator
- Cilj: Kontrola neprimjerenog sadržaja
- Sudionici: baza podataka
- Preduvjet: postoji objavljeni sadržaj
- **Opis osnovnog tijeka:**
 1. Sučelje za administratora nudi pregled objavljenog sadržaja
 2. Administrator pretražuje trenera ili sadržaj koji želi pregledati
 3. Administrator odabire sadržaj
 4. Administrator odlučuje želi li izbrisati cijeli sadržaj ili dio koji je neprimjeren
 5. Administrator klikne na gumb završi
 6. Sustav traži potvrdu
 7. Administrator daje potvrdu
 8. Sustav šalje potvrdu o uspješnosti
- **Opis mogućih odstupanja:**
 1. Trener (korisnik) već izbrisao sadržaj - sustav obavještava administratora

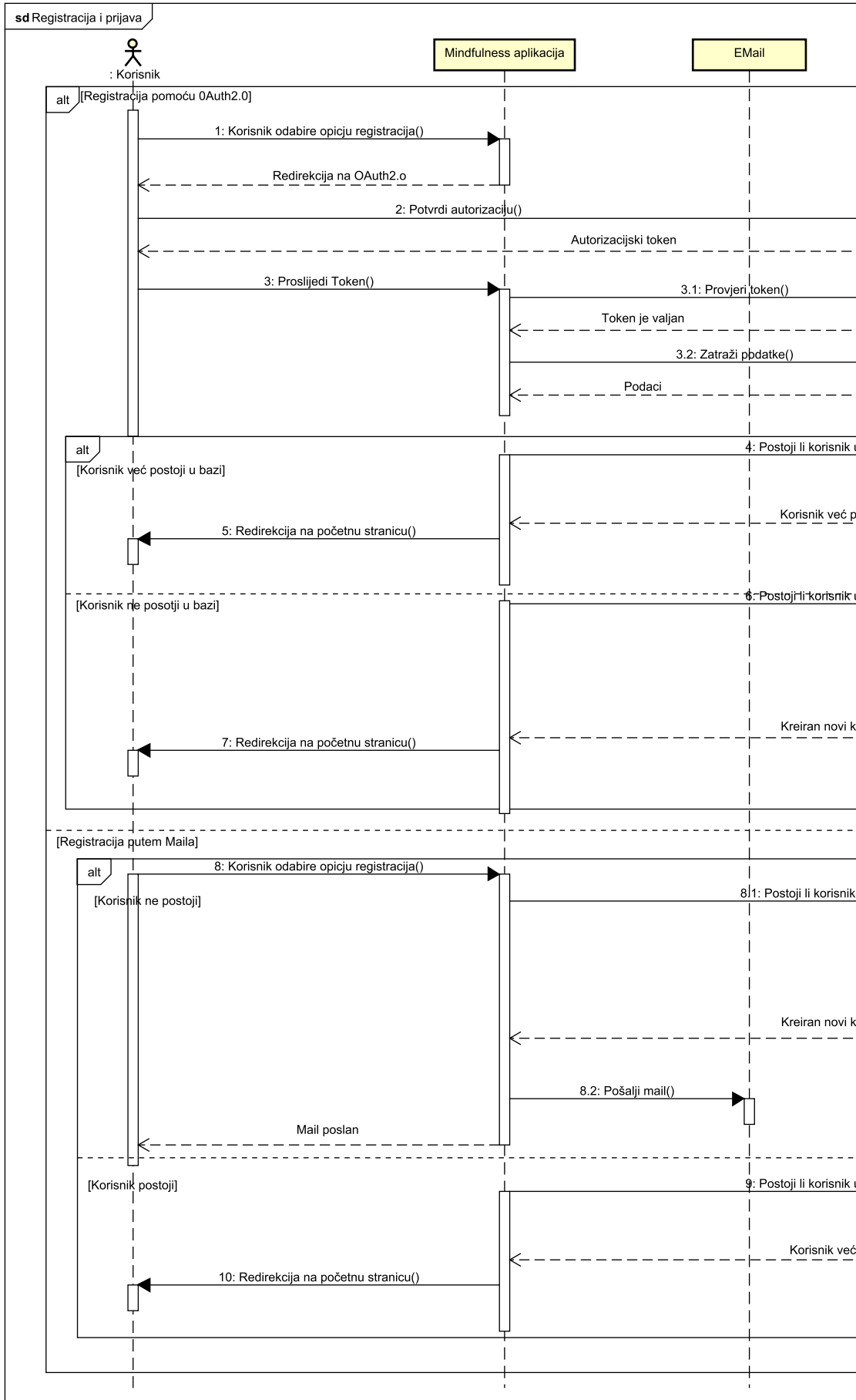
Sekvencijski dijagrami

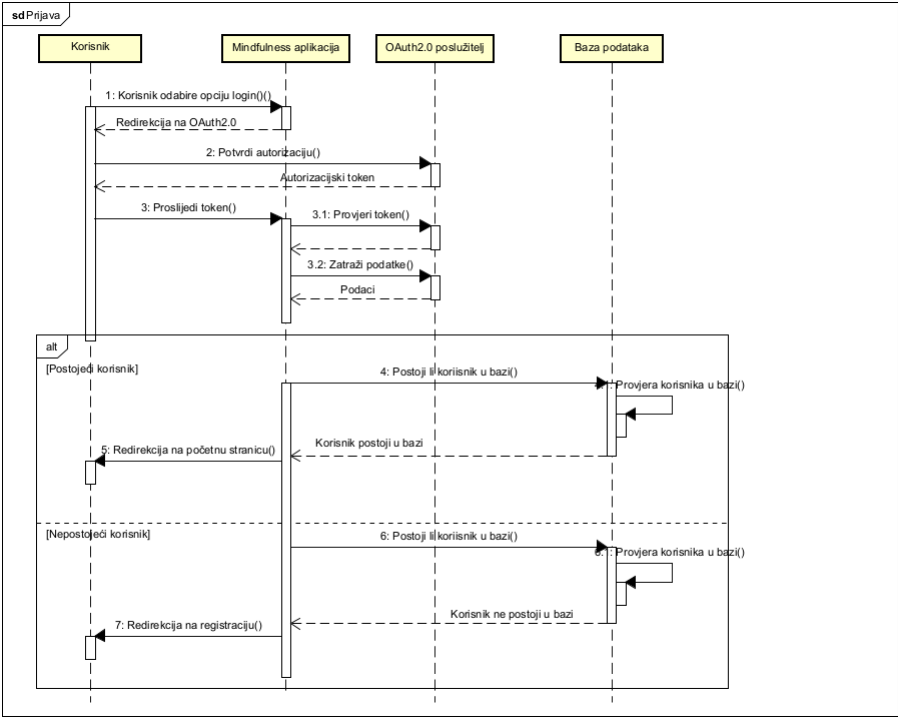
Registracija i prijava korisnika

Ovaj sekvencijski dijagram prikazuje proces registracije i prijave korisnika u web aplikaciju Mindfulness. Proces započinje kada korisnik odabere opciju "Registracija" te unosi svoje podatke nakon čega aplikacija provjerava postoji li korisnik u bazi podataka. Ako korisnik ne postoji, podaci se spremaju u bazu i stvara se novi korisnik.

Alternativno, korisnik može odabrati registraciju putem OAuth 2.0 sustava, pri čemu aplikacija preusmjerava korisnika na poslužitelj za autentifikaciju. Nakon što korisnik odobri pristup, OAuth 2.0 poslužitelj vraća autorizacijski token koji aplikacija provjerava i koristi za dohvat korisničkih podataka. Ako korisnik već postoji, prijava se automatski dovršava i korisnik se preusmjerava u aplikaciju; ako ne postoji, aplikacija kreira novi korisnički račun.

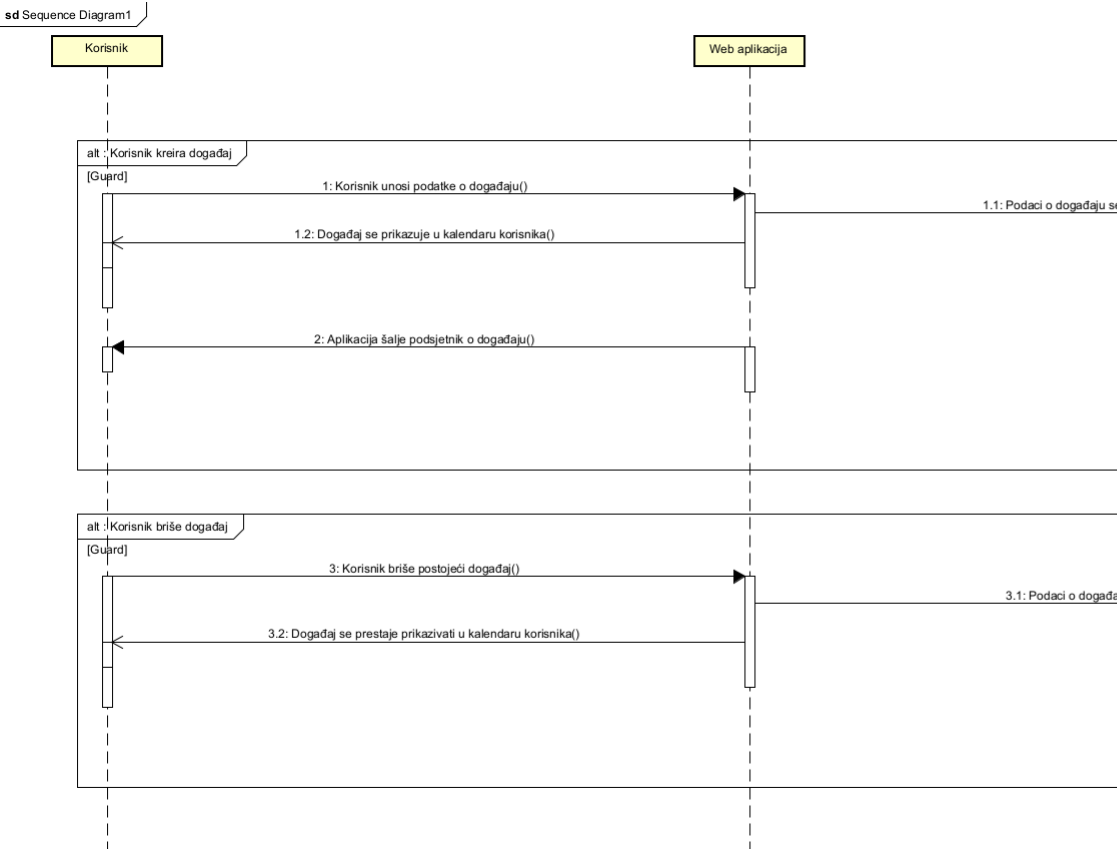
Nakon uspješne registracije, korisnik je usmjeren na glavno sučelje aplikacije.



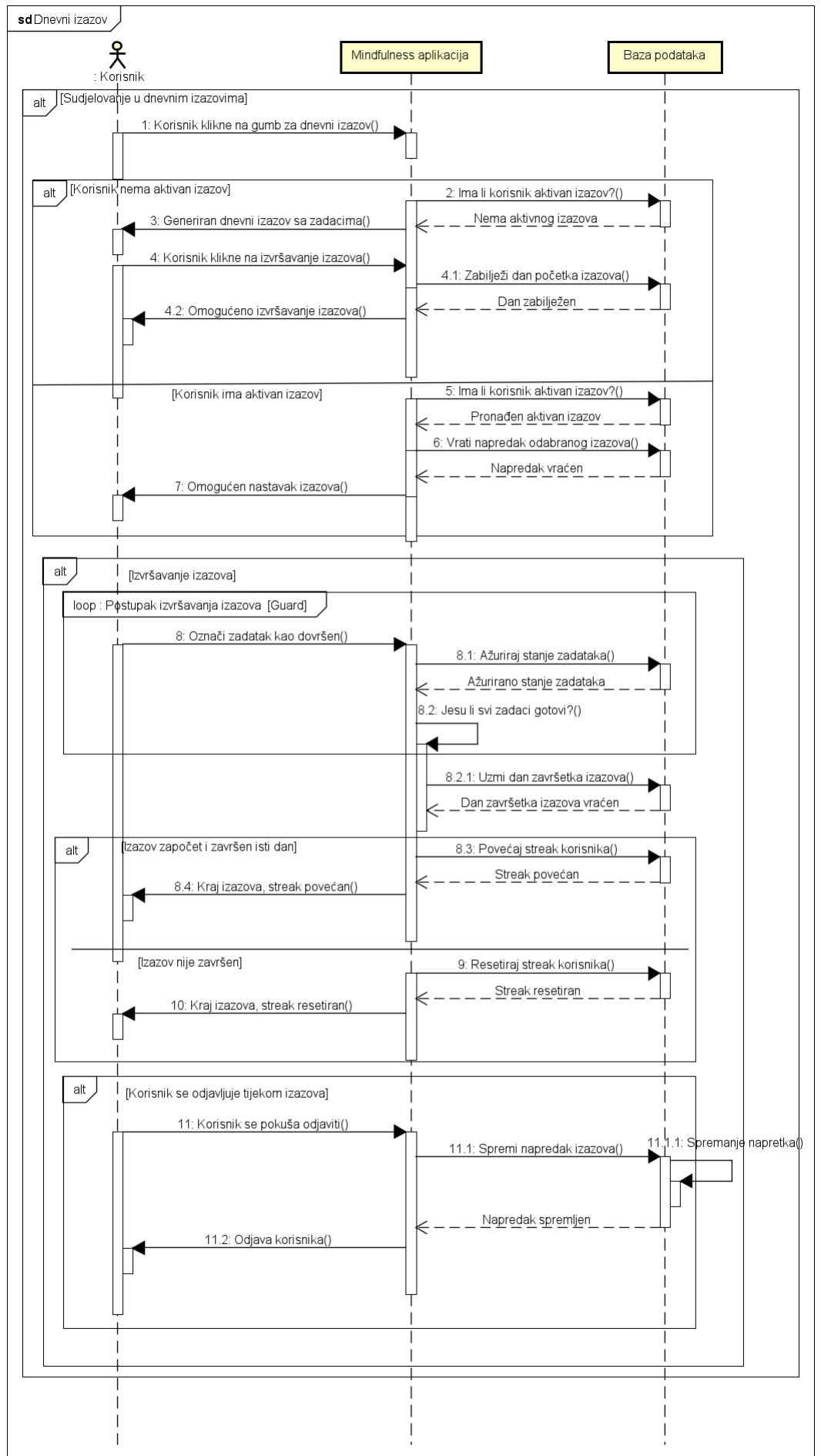


Interakcija korisnika s kalendarom

Ovaj sekvencijski dijagram prikazuje kako korisnik može dodavati i brisati događaje iz svog kalendara.

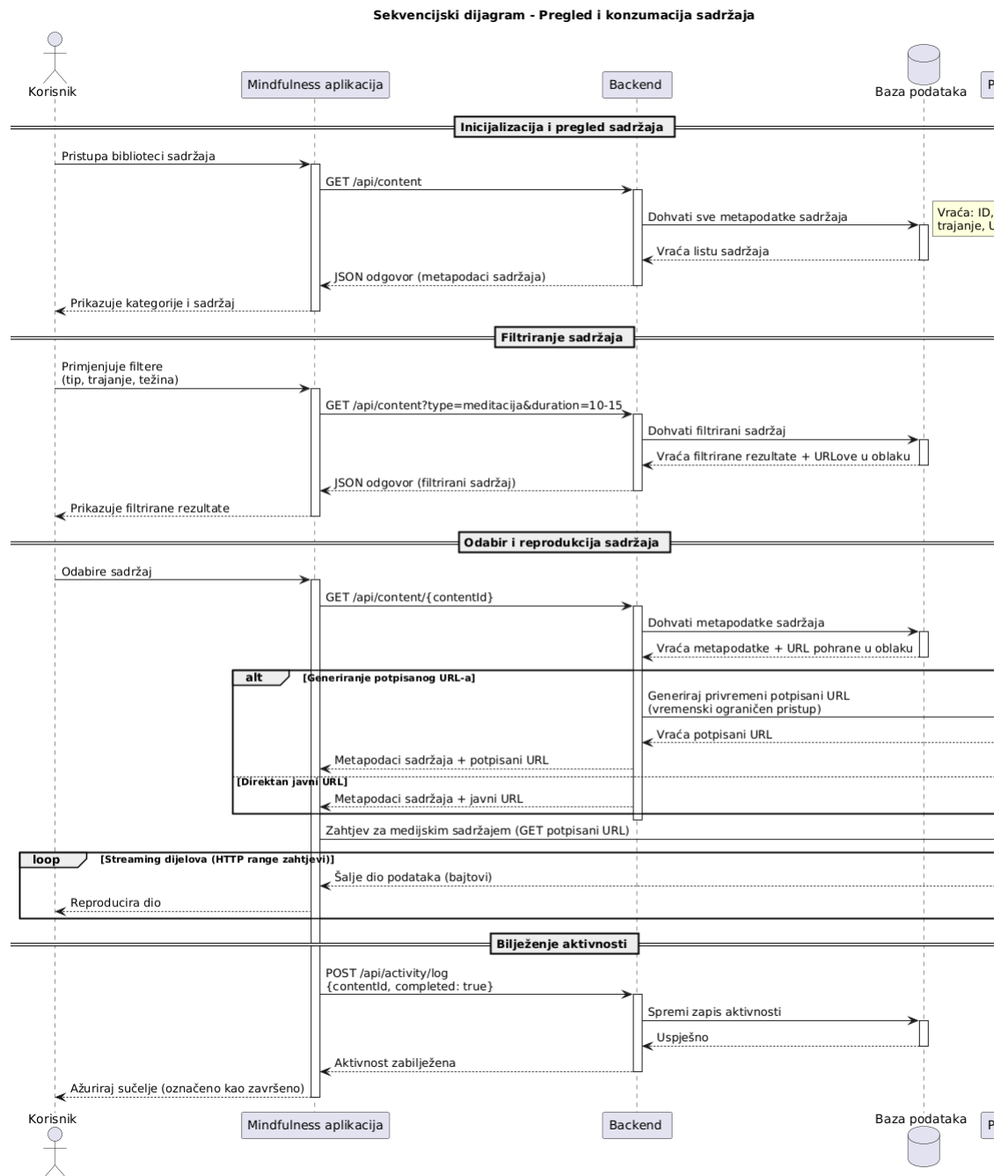


Funkcionalnost dnevnog izazova



Pregled i konzumacija sadržaja

Ovaj sekvencijski dijagram prikazuje interakciju korisnika s dostupnim sadržajem na Mindfulness aplikaciji



Provjera uključenosti ključnih funkcionalnosti u obrasce uporabe

UC id Funkcionalnosti

UC01 F-001, F-010

UC01.1 F-003

UC02 F-001

UC02.1 F-010

UC02.2 F-002

UC02.3 F-001

UC03 F-015, F-015.1

UC03.1 F-015.1

UC04 F-005, F-005.1, F-008

UC05.1 F-011

UC05.2 F-011

UC05.3 F-011

UC05.4 F-012

UC06 F-004

UC07.1 F-009

UC07.2 F-009

UC07.3 F-009

UC07.4 F-009

UC08 F-006

UC08.1 F-007

UC08.2 F-007

UC08.3 F-013

UC id Funkcionalnosti

UC09 F-012, F-014
UC09.1 F-012, F-014
UC10 F-012, F-014
UC11.1 F-012
UC11.2 F-012
UC12 F-012

Arhitektura sustava

Opis arhitekture

Odabrani je stil arhitekture za platformu Mindfulness Model-View-Controller (MVC) arhitektura. Ovakav pristup omogućuje strukturirano razdvajanje funkcionalnosti sustava na zasebne slojeve, čime se postiže veća preglednost i učinkovitost u razvoju i održavanju aplikacije. Razdvajanjem klijentskog, aplikacijskog te sloja baze podataka omogućeno je skaliranje svakog sloja neovisno jedno o drugome. Višeslojna arhitektura također omogućuje dodatne sigurnosne mjere, kao što je npr. zaštita aplikacijskog sloja kroz poslužitelje za autorizaciju i dvofaktorsku autentifikaciju. MVC arhitektura omogućava timovima paralelan rad na sučelju.

Podsustavi

1. Podsustav za autentifikaciju i autorizaciju koji omogućava prijavu i registraciju korisnika. Korisiti OAuth 2.0 za prijavu putem vanjskih identitetskih davatelja.
2. Podsustav za slanje elektroničke pošte, koji omogućava slanje: obavijesti, poruka za zaboravljene lozinke i poruka za potvrđivanje elektroničke pošte.
3. Podsustav za mapiranje DTO objekata na odgovarajuće modele.
4. Podsustav za rad sa bazom podataka, koji omogućava stvaranje, čitanje, ažuriranje i brisanje iz baze podataka.

Spremišta podataka

Podaci će se pohranjivati u relacijskoj bazi podataka koristeći PostgreSQL bazu podataka, koja je pogodna za razvojne potrebe. PostgreSQL baza omogućuje nam učinkovito pohranjivanje strukturiranih podataka poput korisničkih profila, izazova, sadržaja (meditacije, podcasti, članci) itd.

Mrežni protokoli

Za komunikaciju između klijentske i serverske strane koristit će se HTTP protokol. Ovaj protokol omogućuje jednostavan prijenos podataka i osigurava stabilnost pri rukovanju velikim brojem zahtjeva.

Sigurnosni mehanizmi

Aplikacija koristi JSON Web Token za verifikaciju korisnika i za sprječavanje neovlaštenog pristupa podacima. Aplikacija koristi OAuth 2.0 protokol autorizacije.

Globalni upravljački tok

Korisnik započinje interakciju registracijom ili prijavom. Nakon prijave korisnik dobiva sedmodnevni plan praćenja mindfulness vježbi. Korisnik može pregledati druge sadržaje, unositi unos alkohola, kofeina, količinu sna, pratiti svoje raspoloženje itd. Podaci koje korisnik unese šalju se serveru preko HTTP zahtjeva. Zahtjevi se obrađuju u određenom podsustavu, te se korisniku kroz HTTP odgovor vraćaju rezultati i/ili personalizirani sadržaj.

Načini razmjene podataka

Poslužitelj koristi API endpointove da bi odlučio hoće li spremiti podatke koje je korisnik unio u bazu podataka ili će dohvatiti podatke koje je korisnik već unio. API endpointovi su definirani pomoću controllera i [HttpGet(,"")] i [HttpPost(,"")] atributa, gdje između navodnika idu parametri rute. Za unos podataka se koriste metode koje čitaju podatke iz tijela HTTP POST zahtjeva ([FromBody] atribut) i nakon provjere validnosti podataka (npr. postoji li korisnik čije podatke unosimo) unose iste u bazu pomoću DTO-a (Data Transfer Object) koji se mapiraju na objekt u bazi. Dohvat podataka funkcionira na vrlo sličan način, koristi se [HttpGet], i funkcionira u suprotnom smjeru.

Sklopovskoprogramski zahtjevi

Tehnički zahtjevi za našu platformu uključuju podršku za operativne sustave na poslužiteljskoj strani i klijentskoj strani (Windows server, Linux). Poslužitelj mora imati dovoljno resursa za podršku aplikacijskog sloja i baze podataka. Klijentska strana treba moderan web preglednik. ...

Obrazloženje odabira arhitekture

Za Mindfulness platformu odabrali smo MVC (Model-View-Controller) arhitekturu. Ovakav pristup, zbog slojevitosti, omogućuje nam jasnu podjelu između korisničkog sučelja i servera koji obrađuje podatke. MVC arhitektura također omogućuje skalabilnost i fleksibilnost sustava, tj. omogućuje nam fleksibilno dodavanje ili uklanjanje serverskih resursa ili posrednika. Principi oblikovanja koje smo koristili prilikom odabira arhitekture su:

- **Separacija:** jasna separacija među slojevima omogućava donekle neovisne slojeve i paralelan rad na svakom od slojeva.
- **Sigurnost:** centralizirana kontrola podataka na poslužitelju omogućuje jednostavnije upravljanje sigurnošću i autentifikacijom korisnika. U našem je sustavu osigurano da podaci ostaju zaštićeni na poslužitelju. Svaki zahtjev koji klijent šalje poslužitelju mora sadržavati sve informacije potrebne za obradu.
- **Održivost:** zbog jasne podjele, olakšava se održavanje sustava jer se ažuriranja mogu provoditi na poslužitelju bez potreba za izmjenama na klijentima. To nam omogućava kontinuirani razvoj i dodavanje novih značajki uz

minimalne smetnje korisnicima.

- **Skalabilnost:** ovakva arhitektura omogućuje nam skalabilnost od samog početka. To znači da ako se broj korisnika poveća možemo dodati dodatne poslužiteljske resurse kako bi se održala performansa platforme. Isto tako, svaki se podsustav može proširiti i/ili nadograditi bez potrebe za značajnim izmjenama u ostalim dijelovima sustava.
- **Otpornost na greške:** Zbog izolacije, pogreška u jednoj komponenti ne utječe na cijeli sustav.

Organizacija sustava na visokoj razini

- **Klijent-poslužitelj:** Mindfulness platforma kao klijent-poslužitelj arhitektura omogućava centraliziranu obradu i upravljanje podacima na poslužitelju, dok klijenti(korisnici) pristupaju uslugama putem svojih uređaja. Klijentski dio sustava predstavlja aplikaciju koju koriste svi koji žele prakticirati mindfulness u svakodnevnom životu. Klijenti pristupaju funkcionalnostima aplikacije, poput registracije i prijave, pregledavanja i pristupanja meditacijskim vježbama, podcastovima, člancima, formiranja novih materijala za mindfulness itd itd. Poslužiteljski dio fokusira se na obradu zahtjeva klijenata i komuniciranje s bazom podataka kako bi pohranjivao i dohvaćao potrebne informacije.
- **Baza podataka:** Za Mindfulness platformu odabrali smo relacijsku bazu podataka PostgreSQL koja omogućava centraliziranu i strukturiranu pohranu podataka vezanu uz korisničke profile, videe, daily check-inove. Relacijska baza podataka nudi prednosti poput dosljednosti podataka i mogućnosti kreiranja složenih upita, što je prikladno za sustav koji mora osigurati pouzdane i točne informacije. Baza podataka pohranjuje sve relevantne podatke o korisnicima (npr. osobne podatke, interese, lokaciju), detalje o masovnim instrukcijama i grupama za učenje te povijest komunikacije između korisnika
- **Datotečni sustav:** Ako sustav koristi datotečni sustav za pohranu podataka, objasnite njegovu ulogu i način integracije. Datotečni sustav služi kao alat za pohranu video i/ili audio zapisa. Datotečni sustav omogućuje jednostavnu integraciju sa serverom, čime osigurava efikasan pristup dokumentima i olakšava upravljanje većim količinama podataka. Dokumenti su pohranjeni na poslužitelju i povezani su s odgovarajućim korisničkim računima i sesijama instrukcija.
- **Grafičko sučelje:** grafičko sučelje Mindfulness platforme izrađeno je kao web aplikacija koja je dostupna putem internetskog preglednika. To omogućuje pristup platformi s različitih uređaja. Naša web aplikacija pruže intuitivno i responzivno sučelje za jednostavno korištenje glavnih funkcionalnosti. Grafičko sučelje povezano je s poslužiteljem putem HTTP protokola, a zahtjevi se šalju poslužitelju koji obrađuje podatke i vraća rezultate koje sučelje prikazuje korisniku

Organizacija aplikacije

- Frontend sloj implementiran je kao web aplikacija koja služi kao sučelje za interakciju s korisnicima. Ovaj sloj omogućuje korisnicima pristup funkcionalnostima kao što su registracija i prijava, pregled dostupnih medijskih sadržaja, ispunjavanje raspoloženja i mnogim drugima. Frontend sloj koristi HTML, CSS i TypeScript (framework React) za stvaranje responzivnog korisničkog sučelja, a komunikacija s backend slojem odvija se putem REST API-ja.
- Backend sloj odgovoran je za poslovnu logiku i obradu podataka. Na ovom sloju implementirane su funkcionalnosti autentifikacije i autorizacije korisnika (zasad samo to od neceg konktetnog). Backend sloj razvijen je u JetBrains Rider-u. Frontend i backend slojevi međusobno komuniciraju putem HTTP protokola. Korisnički zahtjevi šalju se iz frontend sloja prema backendu, gdje se obrađuju i vraćaju odgovori s podacima.
- MVC arhitektura

Mindfulness platforma organizirana je prema MVC (Model-View-controller) arhitekturi koja omogućuje jasnu raspodjelu odgovornosti

Model: predstavlja podatke i poslovnu logiku aplikacije. Model sloj , također je, zadužen za pristup bazi podataka, dohvat i/ili spremanje informacija.

View: odnosi se na prikaz korisničkog sučelja. U Mindless platformi, frontend sustav služi kao View, preuzimajući podatke iz backenda i prikazujući ih korisniku.

Controller: ovaj sloj povezuje Model i View. Obrađuje zahtjeve korisnika te poziva odgovarajuće metode u Modelu kako bi dobio potrebne podatke.

Baza podataka

Baza podataka našem sustavu pruža strukturirajuću platformu za modeliranje elemenata iz stvarnog svijeta. Osnovni građevni blok baze je entitet, odnosno tablica definirana svojim imenom i skupom atributa. Svrha baze omogućiti je jednostavnu i brzu pohranu, promjenu i dohvaćanje podataka.

Za potrebe naše platforme unutar baze definiramo entitete: AUDIO_LANG, CHALLENGE, CONTENT, CONT_CATEGORY, DAILY_CHECKIN, EVENT, MOTIVATION, REVIEW, SETTINGS, STRT_QUESTION, USER, USER_SETTING

Opis tablica

Svaku tablicu je potrebno opisati po zadanom predlošku. Lijevo se nalazi točno ime varijable u bazi podataka, u sredini se nalazi tip podataka, a desno se nalazi opis varijable. Označite primarni ključ i strani ključ.

AudioLanguage

Atribut	Tip podatka	Opis varijable
Id	GUID	Jedinstveni identifikator jezika (PK)
Name	VARCHAR (30)	Ime jezika

Challenge

Atribut	Tip podatka	Opis varijable
Id	GUID	Jedinstveni identifikator izazova (PK)
Title	VARCHAR(30)	Naziv izazova (obavezan)
Description	VARCHAR(300)	Kratki opis izazova (neobavezan)
Difficulty	ENUM	Zahtjevnost izazova
Duration	INTERVAL	Trajanje izazova
CreatedAt	TIMESTAMP	Datum i vrijeme kada je izazov kreiran

Content

Atribut	Tip podatka	Opis varijable
Id	GUID	Jedinstveni identifikator sadržaja (PK)
Title	VARCHAR(30)	Naziv sadržaja (obavezan)
Description	VARCHAR(300)	Opis sadržaja (neobavezan)
Difficulty	ENUM	Zahtjevnost sadržaja
Duration	INTERVAL	Trajanje sadržaja
UploadedAt	TIMESTAMP	Datum i vrijeme kada je sadržaj objavljen
UserId	GUID	ID korisnika koji je objavio sadržaj (FK)
CategoryId	GUID	ID kategorije kojoj prirpada (FK)
AudioLanguageId	GUID	ID jezika na kojem je (FK)

ContentCategory

Atribut	Tip podatka	Opis varijable
id	GUID	Jedinstveni identifikator jezika (PK)
name	VARCHAR (30)	Ime jezika

Daily checkin

Atribut	Tip podatka	Opis varijable
Id	GUID	Jedinstveni identifikator dnevnog check-ina (PK)
CreatedAt	DATETIMEOFFSET	Datum i vrijeme kreiranja
Mood	INT	Ocjena raspoloženja (neobavezan)
PhysicalActivity	INT	Fizička aktivnost (neobavezan)
Caffeine	INT	Količina unosa kofeina (neobavezan)
SleepScore	INT	Ocjena sna (neobavezan)
Alcohol	INT	Količina alkohola (neobavezan)
DailyNotes	TEXT	Dnevne bilješke (neobavezan)
UserId	GUID	ID korisnika (FK)

Event

Atribut	Tip podatka	Opis varijable
Id	GUID	Jedinstveni identifikator događaja
Title	VARCHAR(30)	Naslov događaja
Description	VARCHAR(300), NULL	Opis događaja
StartTime	DATETIMEOFFSET	Vrijeme početka događaja
EndTime	DATETIMEOFFSET	Vrijeme završetka događaja
UserId	GUID	ID korisnika koji je kreirao događaj (FK)

Motivation

Atribut	Tip podatka	Opis varijable
Id	GUID	Jedinstveni identifikator motivacije
Message	VARCHAR (300)	poruka motivacije

Review

Atribut	Tip podatka	Opis varijable
Id	GUID	Jedinstveni identifikator recenzije (PK)
Rating	INT	Ocjena sadržaja
Comment	VARCHAR(300)	Komentar recenzije (neobavezno)
CreatedAt	DATETIMEOFFSET	Datum i vrijeme kreiranja
UserId	GUID	ID korisnika koji je napisao recenziju (FK)
ContentId	GUID	ID sadržaja koji se recenzira (FK)

Setting

Atribut	Tip podatka	Opis varijable
Id	GUID	Jedinstveni identifikator postavke (PK)
Name	VARCHAR(30)	Naziv postavke
Description	VARCHAR(300), NULL	Opis postavke

Strt_question

Atribut	Tip podatka	Opis varijable
Id	GUID	Jedinstveni identifikator upitnika (PK)
PFocus	INT	Ocjena fokusa
PSleep	INT	Ocjena sna
PBreathing	INT	Ocjena disanja
PStress	INT	Ocjena stresa
PAnxiety	INT	Ocjena anksioznosti
PGratefulness	INT	Ocjena zahvalnosti
PuserId	GUID	ID korisnika (FK)

User

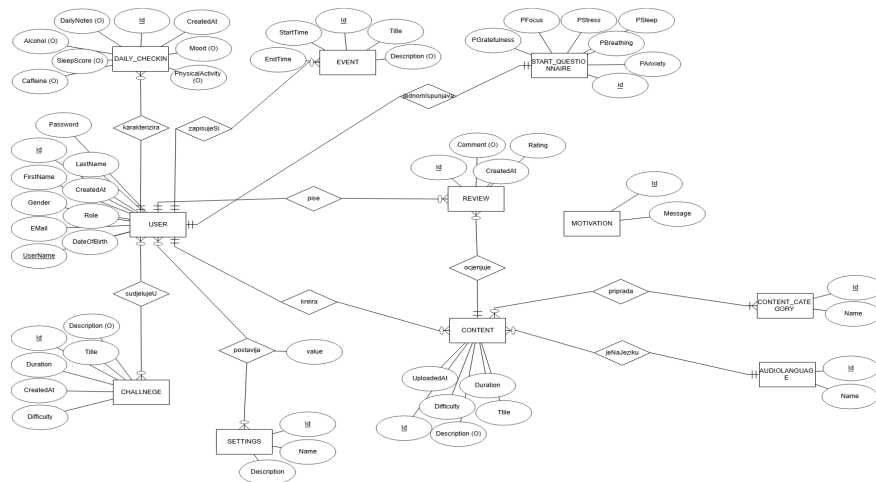
Atribut	Tip podatka	Opis varijable
Id	GUID	Jedinstveni identifikator korisnika (PK)
FirstName	VARCHAR(30)	Ime fokusa
LastName	VARCHAR(30)	Prezime Korisnika
Gender	ENUM	Spol korisnika
DateOfBirth	TIMESTAMP	Datum rođenja korisnika
CreatedAt	INT	Ocjena anksioznosti
StartQuestionnaireId	GUID	ID početnog upitnika kojeg je korisnik rješavao (FK)
PasswordHash	VARCHAR()	Sifra korisnika
Email	GUID	Email korisnika
UserName	VARCHAR(30)	Korisničko ime korisnika

UserSetting

Atribut	Tip podatka	Opis varijable
UserId	GUID	Jedinstveni identifikator korisnika (PK) (FK)
SettingId	VARCHAR(30)	Ime fokusa (PK) (FK)
Value	VARCHAR(30)	Prezime Korisnika

Dijagram baze podataka

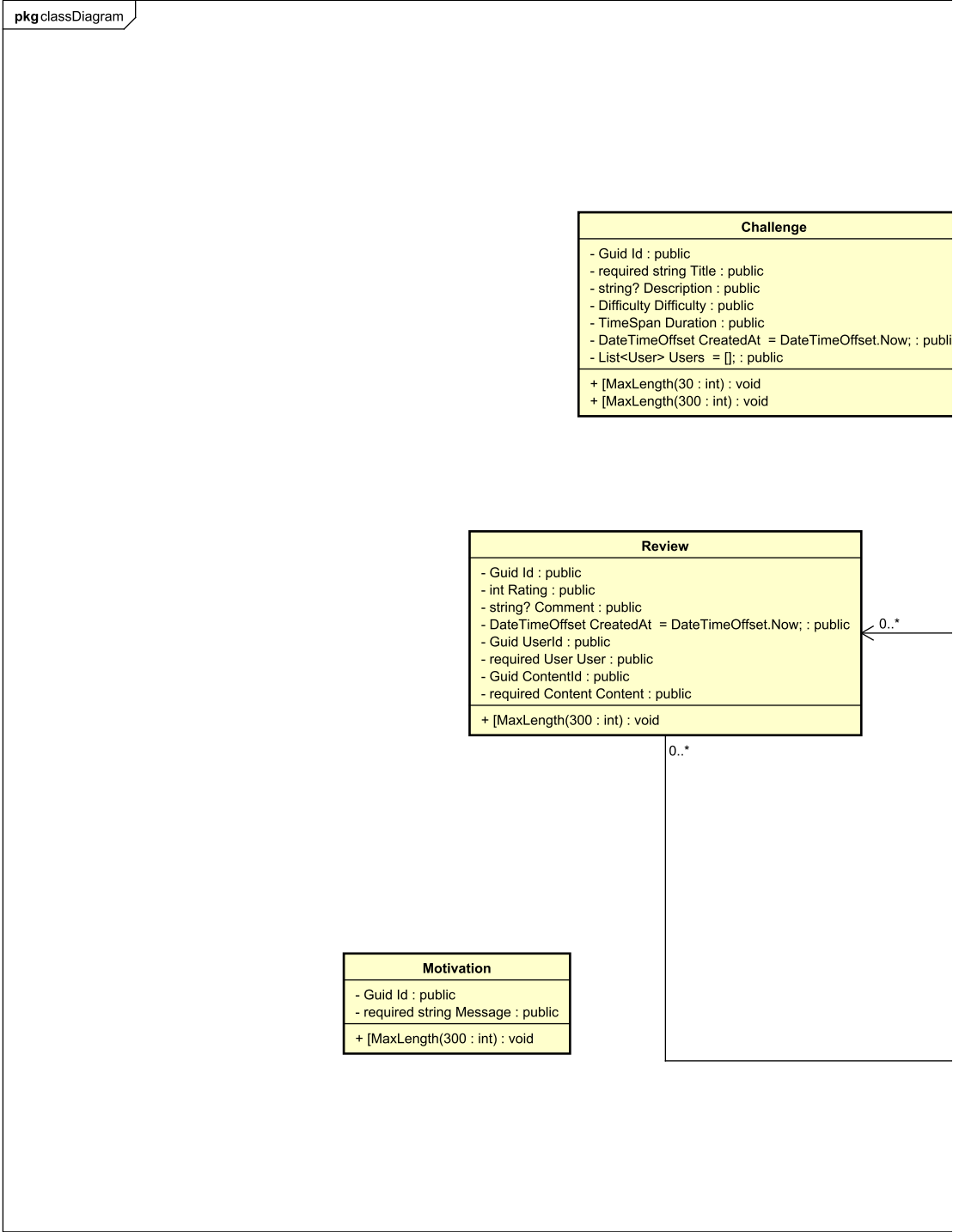
| dijagrami baze podataka



Dijagram razreda

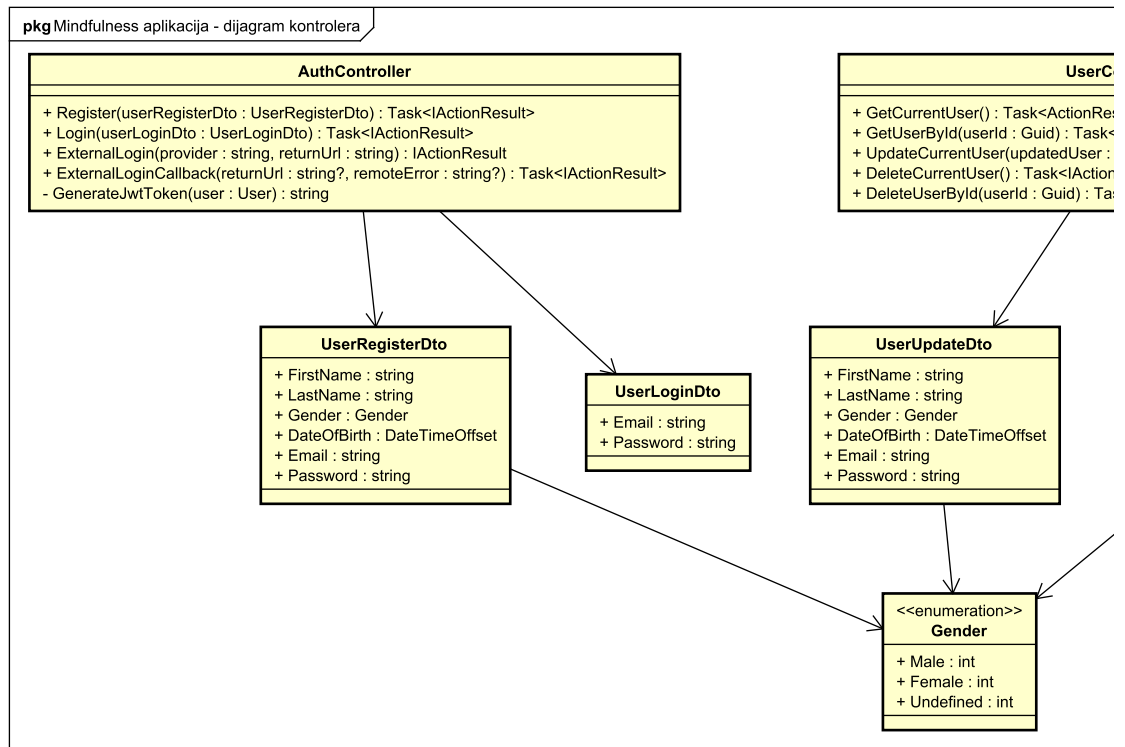
Dijagram klasa

| slika dijagrama klasa



Dijagram kontrolera

| slika dijagrama kontrolera



dio 1. revizije

Prilikom prve predaje projekta, potrebno je priložiti potpuno razrađen dijagram razreda vezan uz generičku funkcionalnost sustava. Ostale funkcionalnosti trebaju biti idejno razrađene u dijagramu sa sljedećim komponentama: nazivi razreda, nazivi metoda i vrste pristupa metodama (npr. javni, zaštićeni), nazivi atributa razreda, veze i odnosi između razreda.

Dinamičko ponašanje aplikacije

Dinamičko ponašanje aplikacije odnosi se na način na koji objekti u sustavu evoluiraju kroz vrijeme, uključujući prijelaze između različitih stanja. To uključuje aktivnosti, događaje, odluke i interakcije unutar aplikacije. UML dijagrami stanja omogućuju vizualizaciju tih promjena i olakšavaju razumijevanje dinamike sustava.

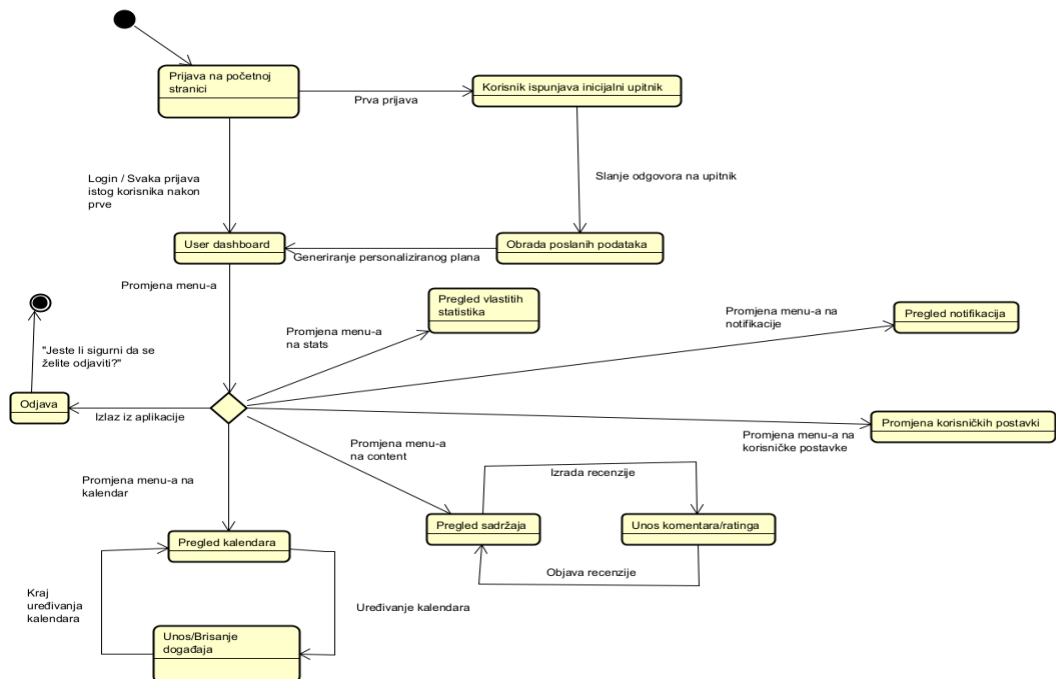
Razumijevanje promjena stanja neophodno je za pravilno funkcioniranje aplikacije jer pruža uvid u interakcije među objektima, komponentama i korisnicima tijekom rada sustava. Korištenjem UML dijagrama stanja i aktivnosti moguće je vizualizirati prijelaze i stanja objekata, identificirati potencijalne probleme, osigurati točnu implementaciju te poboljšati komunikaciju među članovima tima.

Zadatak:

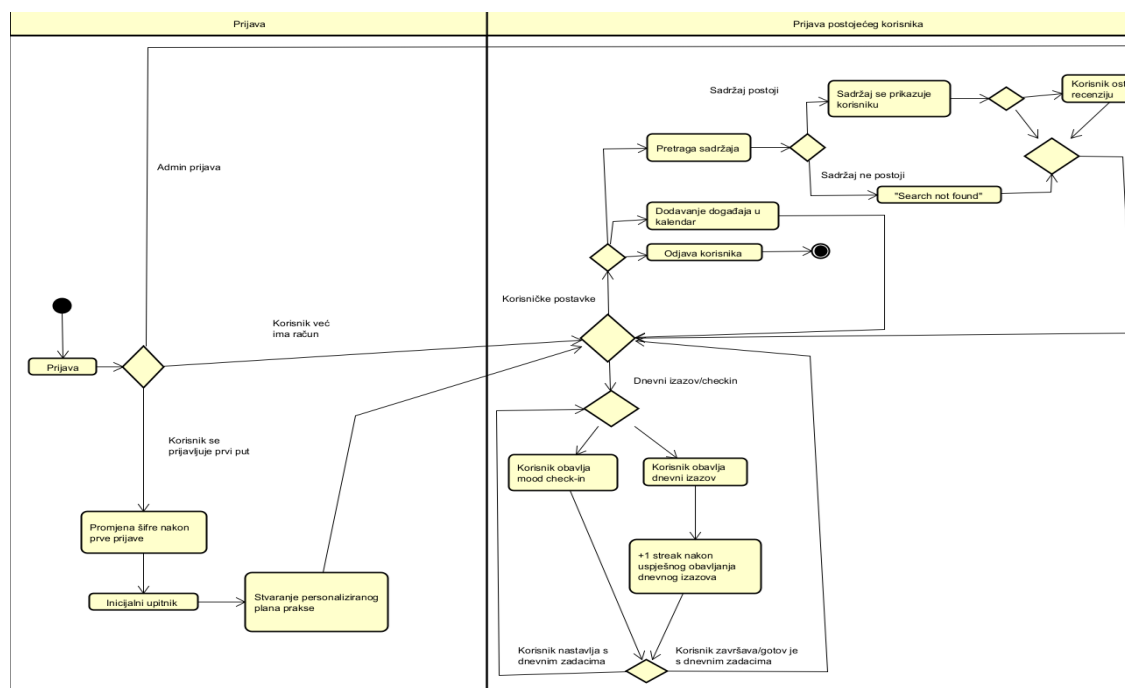
- Dokumentirajte dva dinamička dijagrama koji prikazuju značajne ili neke složene procese u aplikaciji. Registracija i prijava korisnika nisu ključni procesi u ovom kontekstu, stoga se usmjerite na specifične, značajne procese.

UML dijagrami stanja

Pregled vlastitih statistika omogućuje korisniku da vidi prosječni unos kofeina, alkohola, prosječnu duljinu dnevne aktivnosti i sl. Promjena korisničkih postavki omogućuje korisniku da uključi opcionalnu 2FA autentikaciju.

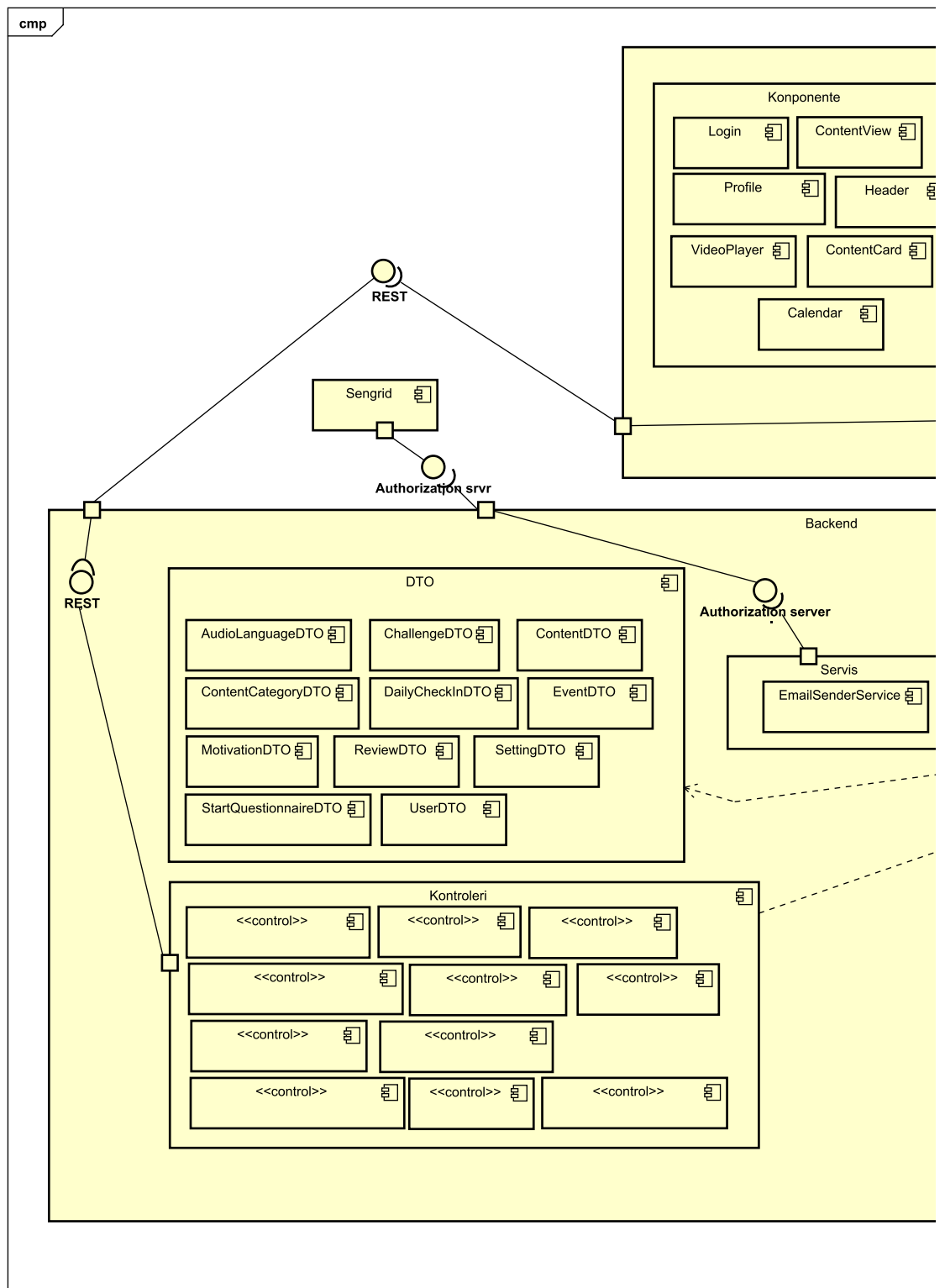


UML dijagrami aktivnosti



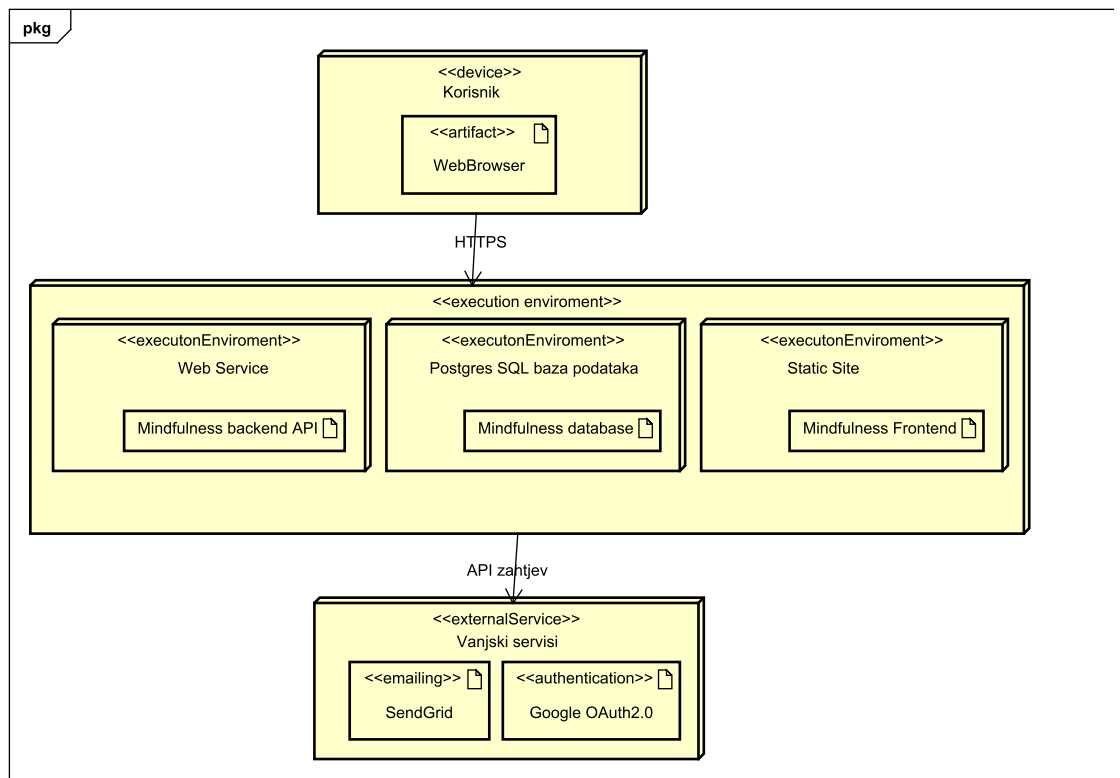
Dijagram komponentata

Ovi dijagrami omogućuju razumijevanje i vizualizaciju fizičkog i logičkog dizajna sustava, uključujući interakcije između dijelova aplikacije.



Dijagram razmještaja

Ovaj dijagram prikazuje razmještaj komponenata softverskog sustava u kojem korisnik putem uređaja ostvaruje interakciju s aplikacijom. U nastavku je specifikacijski dijagram razmještaja.



Ispitivanje komponenti

Cilj ispitivanja komponenti je provjera osnovnih funkcionalnosti implementiranih u razredima sustava.

Sadržaj

- [AuthController - Register Tests](#)
- [AuthController - Login Tests](#)
- [AuthController - ExternalLoginCallback Tests](#)
- [EmailSenderService Tests](#)
- [Sažetak Testnih Slučajeva](#)

1. AuthController - Register Tests

1.1 Register_WithNewUser_ReturnsOk (Redovni slučaj)

Funkcionalnost

Testiranje uspješne registracije novog korisnika u sustav.

Ispitni slučaj

Ulazni podaci:

- Email: `test@example.com`
- Password: `Password123!`
- FirstName: `John`
- LastName: `Doe`

Očekivani rezultati:

- HTTP status: `200 OK`
- Poruka: `"User registration successful."`
- Korisnik kreiran u bazi podataka
- Korisnik dodan u ulogu `"User"`

Dobiveni rezultati: ☐ PASS

Postupak provođenja ispitivanja

- Priprema mock objekata (userManager, signInManager, Mapper, Configuration)
- Postavka mock ponašanja:
 - `FindByEmailAsync` vraća `null` (email ne postoji)
 - `Map<User>` vraća novi User objekt
 - `CreateAsync` vraća `IdentityResult.Success`

- o `AddToRoleAsync` vraća `IdentityResult.Success`

3. Poziv `Register` metode s test podacima
4. Provjera tipa rezultata (`OkObjectResult`)
5. Provjera sadržaja poruke
6. Verifikacija da su sve očekivane metode pozvane

[Test]

```
public async Task Register_WithNewUser_ReturnsOk()

{

    var registerDto = new UserRegisterDto

    {

        Email = "test@example.com",

        Password = "Password123!",

        FirstName = "John",

        LastName = "Doe"

    };

    var user = new User

    {

        Id = Guid.NewGuid(),

        Email = registerDto.Email,

        FirstName = registerDto.FirstName,

        LastName = registerDto.LastName

    };

    _userManagerMock.Setup(x => x.FindByEmailAsync(registerDto.Email))

        .ReturnsAsync((User)null);

    _mapperMock.Setup(x => x.Map(registerDto)).Returns(user);

    _userManagerMock.Setup(x => x.CreateAsync(It.IsAny(), registerDto.Password))

        .ReturnsAsync(IdentityResult.Success);

    _userManagerMock.Setup(x => x.AddToRoleAsync(It.IsAny(), "User"))

        .ReturnsAsync(IdentityResult.Success);

    var result = await _controller.Register(registerDto);

    Assert.That(result, Is.InstanceOf());

    var okResult = result as OkObjectResult;

    Assert.That(okResult.Value, Is.EqualTo("User registration successful.));

}
```

1.2 Register_WithExistingEmail_ReturnsBadRequest (Rubni uvjet)

Funkcionalnost

Testiranje pokušaja registracije s emailom koji je već u upotrebi.

Ispitni slučaj

Ulazni podaci:

- Email: `existing@example.com` (već postoji u sustavu)
- Password: `Password123!`
- FirstName: `John`
- LastName: `Doe`

Očekivani rezultati:

- HTTP status: `400 Bad Request`
- Poruka: `"This email is already in use."`
- Korisnik nije kreiran

Dobiveni rezultati: ☐ PASS

Postupak provođenja ispitivanja

1. Priprema postojećeg korisnika s istim emailom
2. Postavka `FindByEmailAsync` da vraća postojećeg korisnika
3. Poziv `Register` metode
4. Provjera da rezultat nije `OkObjectResult` već `BadRequestObjectResult`
5. Provjera sadržaja poruke o pogrešci
6. Verifikacija da `CreateAsync` nije pozvan

[Test]

```
public async Task Register_WithExistingEmail_ReturnsBadRequest()

{

    var registerDto = new UserRegisterDto

    {

        Email = "existing@example.com",

        Password = "Password123!",

        FirstName = "John",

        LastName = "Doe"

    };

    var existingUser = new User {

        FirstName = "John",

        LastName = "Doe",

        Email = registerDto.Email

    };

    _userManagerMock.Setup(x => x.FindByEmailAsync(registerDto.Email))

        .ReturnsAsync(existingUser);

    var result = await _controller.Register(registerDto);

    Assert.That(result, Is.InstanceOf());

    var badRequestResult = result as BadRequestObjectResult;

    Assert.That(badRequestResult.Value, Is.EqualTo("This email is already in use."));

}
```

1.3 Register_WhenUserCreationFails_ReturnsBadRequest (Izazivanje pogreške)

Funkcionalnost

Testiranje reakcije sustava kada kreiranje korisnika ne uspije (npr. slaba lozinka).

Ispitni slučaj

Ulazni podaci:

- Email: `test@example.com`
- Password: `Password123!`
- FirstName: `John`
- LastName: `Doe`

Očekivani rezultati:

- HTTP status: `400 Bad Request`
- `IdentityError` s opisom: `"Password too weak"`
- Korisnik nije kreiran

Dobiveni rezultati: ☐ PASS

Postupak provođenja ispitivanja

1. Priprema mock objekata
2. Postavka `FindByEmailAsync` da vraća `null`
3. Postavka `CreateAsync` da vraća `IdentityResult.Failed` s porukom "Password too weak"
4. Poziv `Register` metode
5. Provjera da je rezultat `BadRequestObjectResult`
6. Verifikacija da korisnik nije dodan u ulogu

```
[Test]

public async Task Register_WhenUserCreationFails_ReturnsBadRequest()
{
    var registerDto = new UserRegisterDto
    {
        Email = "test@example.com",
        Password = "Password123!",
        FirstName = "John",
        LastName = "Doe"
    };

    var user = new User
    {
        FirstName = "John",
        LastName = "Doe",
        Email = registerDto.Email
    };

    var errors = new[] { new IdentityError { Description = "Password too weak" } };

    _userManagerMock.Setup(x => x.FindByEmailAsync(registerDto.Email))
        .ReturnsAsync((User) null);

    _mapperMock.Setup(x => x.Map(registerDto)).Returns(user);

    _userManagerMock.Setup(x => x.CreateAsync(It.IsAny(), registerDto.Password))
        .ReturnsAsync(IdentityResult.Failed(errors));
```

```
var result = await _controller.Register(registerDto);
```

```
Assert.That(result, Is.InstanceOf());
```

```
}
```

1.4 Register_WhenAddToRoleFails_ReturnsBadRequest (Izazivanje pogreške)

Funkcionalnost

Testiranje reakcije kada dodavanje korisnika u ulogu ne uspije.

Ispitni slučaj

Ulazni podaci:

- Email: test@example.com
- Password: Password123!
- FirstName: John
- LastName: Doe

Očekivani rezultati:

- HTTP status: 400 Bad Request
- IdentityError s opisom: "Role assignment failed"
- Korisnik kreiran, ali nije dodan u ulogu

Dobiveni rezultati: ☐ PASS

Postupak provođenja ispitivanja

1. Postavka CreateAsync da uspije
2. Postavka AddToRoleAsync da vrati IdentityResult.Failed s greškom
3. Poziv Register metode
4. Provjera rezultata je BadRequestObjectResult
5. Verifikacija da je korisnik kreiran, ali dodavanje u ulogu je neuspjelo

```
[Test]
```

```
public async Task Register_WhenAddToRoleFails_ReturnsBadRequest()
```

```
{
```

```
var registerDto = new UserRegisterDto
```

```
{
```

```
Email = "test@example.com",
```

```
Password = "Password123!",
```

```
FirstName = "John",
```

```
LastName = "Doe"
```

```
};
```

```
var user = new User
```

```
{
```

```
FirstName = "John",
```

```
LastName = "Doe"
```

```
};
```

```
var errors = new[] { new IdentityError { Description = "Role assignment failed" } };
```

```
_userManagerMock.Setup(x => x.FindByEmailAsync(registerDto.Email))
```

```
.ReturnsAsync((User)null);
```

```

        _mapperMock.Setup(x => x.Map(registerDto)).Returns(user);

        _userManagerMock.Setup(x => x.CreateAsync(It.IsAny(), registerDto.Password))
            .ReturnsAsync(IdentityResult.Success);

        _userManagerMock.Setup(x => x.AddToRoleAsync(It.IsAny(), "User"))
            .ReturnsAsync(IdentityResult.Failed(errors));

        var result = await _controller.Register(registerDto);

        Assert.That(result, Is.InstanceOf());
    }
}

```

2. AuthController - Login Tests

2.1 Login_WithValidCredentials_ReturnsOkWithToken (Redovni slučaj)

Funkcionalnost

Testiranje uspješne prijave korisnika s validnim pristupnim podacima.

Ispitni slučaj

Ulazni podaci:

- Email: `test@example.com`
- Password: `Password123!`

Očekivani rezultati:

- HTTP status: `200 OK`
- Odgovor sadrži JWT token
- Token nije prazan string

Dobiveni rezultati: ☐ PASS

Postupak provođenja ispitivanja

1. Priprema postojećeg korisnika
2. Postavka `FindByEmailAsync` da vraća korisnika
3. Postavka `CheckPasswordSignInAsync` da vraća `SignInResult.Success`
4. Postavka JWT konfiguracije (Issuer, Audience, Secret Key)
5. Poziv `Login` metode
6. Provjera tipa rezultata (`OkObjectResult`)
7. Ekstrakcija i provjera postojanja tokena pomoću refleksije
8. Verifikacija da token nije null ili prazan

```

[Test]
public async Task Login_WithValidCredentials_ReturnsOkWithToken()
{
    var loginDto = new UserLoginDto
    {
        Email = "test@example.com",
        Password = "Password123!"
    };

    var user = new User
    {

```

```

        Id = Guid.NewGuid(),

        FirstName = "John",

        LastName = "Doe",

        Email = loginDto.Email,

        UserName = loginDto.Email

    };

    _userManagerMock.Setup(x => x.FindByEmailAsync(loginDto.Email))

        .ReturnsAsync(user);

    _signInManagerMock.Setup(x => x.CheckPasswordSignInAsync(user, loginDto.Password, false))

        .ReturnsAsync(Microsoft.AspNetCore.Identity.SignInResult.Success);

    var result = await _controller.Login(loginDto);

    Assert.That(result, Is.InstanceOf());

    var okResult = result as OkObjectResult;

    Assert.That(okResult.Value, Is.Not.Null);

    var tokenProperty = okResult.Value.GetType().GetProperty("token");

    Assert.That(tokenProperty, Is.Not.Null);

    var token = tokenProperty.GetValue(okResult.Value) as string;

    Assert.That(token, Is.Not.Null.And.Not.Empty);

}

```

2.2 Login_WithInvalidEmail_ReturnsUnauthorized (Rubni uvjet)

Funkcionalnost

Testiranje prijave s nepostojećim emailom.

Ispitni slučaj

Ulazni podaci:

- Email: nonexistent@example.com
- Password: Password123!

Očekivani rezultati:

- HTTP status: 401 Unauthorized
- Poruka: "Invalid email."

Dobiveni rezultati: ☐ PASS

Postupak provođenja ispitivanja

1. Postavka `FindByEmailAsync` da vraća `null`
2. Poziv `Login` metode
3. Provjera rezultata je `UnauthorizedObjectResult`
4. Provjera poruke o pogrešci
5. Verifikacija da `CheckPasswordSignInAsync` nije pozvan

```
[Test]
```

```
public async Task Login_WithInvalidEmail_ReturnsUnauthorized()
```

```
{
```

```
    var loginDto = new UserLoginDto
```

```

{
    Email = "nonexistent@example.com",
    Password = "Password123!"
};

_userManagerMock.Setup(x => x.FindByEmailAsync(loginDto.Email))
    .ReturnsAsync((User)null);

var result = await _controller.Login(loginDto);

Assert.That(result, Is.InstanceOf());

var unauthorizedResult = result as UnauthorizedObjectResult;

Assert.That(unauthorizedResult.Value, Is.EqualTo("Invalid email.));
}

```

2.3 Login_WithInvalidPassword_ReturnsUnauthorized (Rubni uvjet)

Funkcionalnost

Testiranje prijave s ispravnim emailom ali pogrešnom lozinkom.

Ispitni slučaj

Ulazni podaci:

- Email: `test@example.com` (postojeći)
- Password: `WrongPassword!` (pogrešna)

Očekivani rezultati:

- HTTP status: `401 Unauthorized`
- Poruka: `"Invalid password."`

Dobiveni rezultati: ☐ PASS

Postupak provođenja ispitivanja

1. Priprema postojećeg korisnika
2. Postavka `FindByEmailAsync` da vraća korisnika
3. Postavka `CheckPasswordSignInAsync` da vraća `SignInResult.Failed`
4. Poziv `Login` metode
5. Provjera rezultata je `UnauthorizedObjectResult`
6. Provjera poruke o pogrešci "Invalid password."

```

[Test]

public async Task Login_WithInvalidPassword_ReturnsUnauthorized()
{
    var loginDto = new UserLoginDto
    {
        Email = "test@example.com",
        Password = "WrongPassword!"
    };

    var user = new User
    {
        Id = Guid.NewGuid(),
        FirstName = "John",

```

```

        LastName = "Doe",

        Email = loginDto.Email

    };

    _userManagerMock.Setup(x => x.FindByEmailAsync(loginDto.Email))
        .ReturnsAsync(user);

    _signInManagerMock.Setup(x => x.CheckPasswordSignInAsync(user, loginDto.Password, false))
        .ReturnsAsync(Microsoft.AspNetCore.Identity.SignInResult.Failed);

    var result = await _controller.Login(loginDto);

    Assert.That(result, Is.InstanceOf());

    var unauthorizedResult = result as UnauthorizedObjectResult;

    Assert.That(unauthorizedResult.Value, Is.EqualTo("Invalid password.")).
}

```

3. AuthController - ExternalLoginCallback Tests

3.1 ExternalLoginCallback_WithRemoteError_ReturnsBadRequest (Izazivanje pogreške)

Funkcionalnost

Testiranje reakcije na grešku od vanjskog pružatelja autentifikacije (Google, Facebook).

Ispitni slučaj

Ulazni podaci:

- remoteError: "Access denied"

Očekivani rezultati:

- HTTP status: 400 Bad Request
- Poruka sadrži: "Access denied"

Dobiveni rezultati: ☒ PASS

Postupak provođenja ispitivanja

- Poziv `ExternalLoginCallback` s parametrom `remoteError`
- Provera tipa rezultata (`BadRequestObjectResult`)
- Provera da poruka sadrži remote error tekst
- Verifikacija da nema daljnje obrade

```

[Test]

public async Task ExternalLoginCallback_WithRemoteError_ReturnsBadRequest()

{

    var remoteError = "Access denied";

    var result = await _controller.ExternalLoginCallback(null, remoteError);

    Assert.That(result, Is.InstanceOf());

    var badRequestResult = result as BadRequestObjectResult;

    Assert.That(badRequestResult.Value, Does.Contain(remoteError));

}

```

3.2 ExternalLoginCallback_WithNoExternalLoginInfo_ReturnsBadRequest (Izazivanje pogreške)

Funkcionalnost

Testiranje situacije kada vanjski login info nije dostupan.

Ispitni slučaj

Ulazni podaci:

- Nema remote error-a
- GetExternalLoginInfoAsync vraća null

Očekivani rezultati:

- HTTP status: 400 Bad Request
- Poruka: "Error loading external login info."

Dobiveni rezultati: ☐ PASS

Postupak provođenja ispitivanja

1. Postavka `GetExternalLoginInfoAsync` da vraća `null`
2. Poziv `ExternalLoginCallback` bez parametara
3. Provjera rezultata je `BadRequestObjectResult`
4. Provjera točne poruke o pogrešci

```
[Test]

public async Task ExternalLoginCallback_WithNoExternalLoginInfo_ReturnsBadRequest()

{

    _signInManagerMock.Setup(x => x.GetExternalLoginInfoAsync())

        .ReturnsAsync((ExternalLoginInfo) null);

    var result = await _controller.ExternalLoginCallback();

    Assert.That(result, Is.InstanceOf());

    var badRequestResult = result as BadRequestObjectResult;

    Assert.That(badRequestResult.Value, Is.EqualTo("Error loading external login info.")).

}
```

3.3 ExternalLoginCallback_WithNoEmailClaim_ReturnsBadRequest (Rubni uvjet)

Funkcionalnost

Testiranje situacije kada vanjski pružatelj ne vraća email claim.

Ispitni slučaj

Ulazni podaci:

- Google login bez email claima
- LoginProvider: "Google"
- ProviderKey: "12345"

Očekivani rezultati:

- HTTP status: 400 Bad Request
- Poruka: "Google account has no email claim."

Dobiveni rezultati: ☐ PASS

Postupak provođenja ispitivanja

1. Kreiranje ClaimsPrincipal bez email claima
2. Kreiranje ExternalLoginInfo objekta s Google providerom
3. Postavka `GetExternalLoginInfoAsync` da vraća taj info
4. Poziv `ExternalLoginCallback`
5. Provjera rezultata i poruke koja uključuje naziv providera

```
[Test]

public async Task ExternalLoginCallback_WithNoEmailClaim_ReturnsBadRequest()

}
```



```

{

    var claims = new List();

    var identity = new ClaimsIdentity(claims);

    var principal = new ClaimsPrincipal(identity);

    var loginInfo = new ExternalLoginInfo(principal, "Google", "12345", "Google");

    _signInManagerMock.Setup(x => x.GetExternalLoginInfoAsync())
        .ReturnsAsync(loginInfo);

    var result = await _controller.ExternalLoginCallback();

    Assert.That(result, Is.InstanceOf());

    var badRequestResult = result as BadRequestObjectResult;

    Assert.That(badRequestResult.Value, Is.EqualTo("Google account has no email claim.));

}

```

3.4 ExternalLoginCallback_WithExistingUser_ReturnsRedirectWithToken (Redovni slučaj)

Funkcionalnost

Testiranje prijave postojećeg korisnika preko vanjskog pružatelja.

Ispitni slučaj

Ulazni podaci:

- Email: `test@example.com` (postojeći korisnik)
- LoginProvider: `"Google"`
- ProviderKey: `"12345"`

Očekivani rezultati:

- HTTP status: `302 Redirect`
- URL sadrži JWT token parametar
- Vanjski login povezan s korisnikom

Dobiveni rezultati: ☐ PASS

Postupak provođenja ispitivanja

1. Kreiranje ClaimsPrincipal s email claimom
2. Kreiranje ExternalLoginInfo objekta
3. Priprema postojećeg korisnika
4. Postavka mock metoda:
 - `GetExternalLoginInfoAsync` vraća login info
 - `FindByEmailAsync` vraća korisnika
 - `FindByLoginAsync` vraća `null` (login još nije povezan)
 - `AddLoginAsync` vraća `Success`
5. Poziv `ExternalLoginCallback`
6. Provjera tipa rezultata (`RedirectResult`)
7. Provjera da URL sadrži "token=" parametar

```
[Test]
```

```

public async Task ExternalLoginCallback_WithExistingUser_ReturnsRedirectWithToken()
{

    var email = "test@example.com";

    var claims = new List

    {

```

```

        new Claim(ClaimTypes.Email, email)

    };

    var identity = new ClaimsIdentity(claims);

    var principal = new ClaimsPrincipal(identity);

}

var loginInfo = new ExternalLoginInfo(principal, "Google", "12345", "Google");

}

var existingUser = new User

{

    Id = Guid.NewGuid(),

    FirstName = "John",

    LastName = "Doe",

    Email = email,

    UserName = email

};

_signinManagerMock.Setup(x => x.GetExternalLoginInfoAsync())

    .ReturnsAsync(loginInfo);

}

_userManagerMock.Setup(x => x.FindByEmailAsync(email))

    .ReturnsAsync(existingUser);

}

_userManagerMock.Setup(x => x.FindByLoginAsync(loginInfo.LoginProvider, loginInfo.ProviderKey))

    .ReturnsAsync((User)null);

}

_userManagerMock.Setup(x => x.AddLoginAsync(existingUser, loginInfo))

    .ReturnsAsync(IdentityResult.Success);

}

var result = await _controller.ExternalLoginCallback();

}

Assert.That(result, Is.InstanceOf());

var redirectResult = result as RedirectResult;

Assert.That(redirectResult.Url, Does.Contain("token="));

}

```

3.5 ExternalLoginCallback_WithNewUser_CreatesUserAndReturnsRedirect (Redovni slučaj)

Funkcionalnost

Testiranje kreiranja novog korisnika prilikom prve prijave preko vanjskog pružatelja.

Ispitni slučaj

Ulazni podaci:

- Email: `newuser@example.com` (ne postoji u sustavu)
- Name: `"John Doe"`
- Surname: `"Doe"`
- LoginProvider: `"Google"`

Očekivani rezultati:

- HTTP status: 302 Redirect
- Novi korisnik kreiran
- Korisnik dodan u ulogu "User"
- Vanjski login povezan s korisnikom
- URL sadrži token

Dobiveni rezultati: ☐ PASS

Postupak provođenja ispitivanja

1. Kreiranje ClaimsPrincipal s email, name i surname claimovima
2. Kreiranje ExternalLoginInfo
3. Postavka mock metoda:
 - FindByEmailAsync vraća null (novi korisnik)
 - CreateAsync vraća Success
 - AddToRoleAsync vraća Success
 - AddLoginAsync vraća Success
4. Poziv ExternalLoginCallback
5. Provjera rezultata je RedirectResult
6. Provjera URL-a sadrži token
7. Verifikacija poziva CreateAsync (Times.Once)
8. Verifikacija poziva AddToRoleAsync (Times.Once)

[Test]

```
public async Task ExternalLoginCallback_WithNewUser_CreatesUserAndReturnsRedirect()
{
    var email = "newuser@example.com";

    var claims = new List
    {
        new(ClaimTypes.Email, email),
        new(ClaimTypes.Name, "John Doe"),
        new(ClaimTypes.Surname, "Doe")
    };

    var identity = new ClaimsIdentity(claims);
    var principal = new ClaimsPrincipal(identity);

    var loginInfo = new ExternalLoginInfo(principal, "Google", "12345", "Google");

    _signInManagerMock.Setup(x => x.GetExternalLoginInfoAsync())
        .ReturnsAsync(loginInfo);

    _userManagerMock.Setup(x => x.FindByEmailAsync(email))
        .ReturnsAsync((User) null);

    _userManagerMock.Setup(x => x.CreateAsync(It.IsAny()))
        .ReturnsAsync(IdentityResult.Success);

    _userManagerMock.Setup(x => x.AddToRoleAsync(It.IsAny(), "User"))
        .ReturnsAsync(IdentityResult.Success);
}
```

```

        _userManagerMock.Setup(x => x.AddLoginAsync(It.IsAny(), loginInfo))
            .ReturnsAsync(IdentityResult.Success);

        var result = await _controller.ExternalLoginCallback();

        Assert.That(result, Is.InstanceOf());

        var redirectResult = result as RedirectResult;

        Assert.That(redirectResult.Url, Does.Contain("token="));

        _userManagerMock.Verify(x => x.CreateAsync(It.IsAny()), Times.Once);

        _userManagerMock.Verify(x => x.AddToRoleAsync(It.IsAny(), "User"), Times.Once);
    }

```

4. EmailSenderService Tests

4.1 SendEmailAsync_WithValidParameters_SendsEmailSuccessfully (Redovni slučaj)

Funkcionalnost

Testiranje uspješnog slanja emaila sa svim validnim parametrima.

Ispitni slučaj

Ulazni podaci:

- toEmail: test@example.com
- subject: "Test Subject"
- message: "Test Message Content"
- API Key: "SG.test-api-key-12345"

Očekivani rezultati:

- Email uspješno poslan
- From adresa: support@progimindfulness.app
- From ime: "Support"
- Subject: "Test Subject"
- PlainTextContent: "Test Message Content"
- HtmlContent: "Test Message Content"

Dobiveni rezultati: ☐ PASS

Postupak provođenja ispitivanja

1. Postavljanje environment varijable SEND_GRID_KEY
2. Kreiranje TestableEmailSenderService s HttpStatusCode.OK
3. Poziv SendEmailAsync metode
4. Provjera da metoda ne baca iznimku
5. Provjera LastSentMessage objekta:
 - From email i name
 - Subject
 - PlainTextContent i HtmlContent
6. Korištenje Assert.Multiple za grupiranje povezanih provjera

```

[Test]
public async Task SendEmailAsync_WithValidParameters_SendsEmailSuccessfully()
{
    var service = new TestableEmailSenderService(HttpStatusCode.OK);

    Assert.DoesNotThrowAsync(async () =>
    {
        await service.SendEmailAsync(TestEmail, TestSubject, TestMessage);
    });
}

```

```

        Assert.That(service.LastSentMessage, Is.Not.Null);

        Assert.Multiple(() =>
        {
            Assert.That(service.LastSentMessage.From.Email, Is.EqualTo("support@progimindfulness.app"));

            Assert.That(service.LastSentMessage.From.Name, Is.EqualTo("Support"));

            Assert.That(service.LastSentMessage.Subject, Is.EqualTo(TestSubject));

            Assert.That(service.LastSentMessage.PlainTextContent, Is.EqualTo(TestMessage));

            Assert.That(service.LastSentMessage.HtmlContent, Is.EqualTo(TestMessage));

        });
    }
}

```

4.2 SendEmailAsync_WithNullApiKey_ThrowsException (Izazivanje pogreške)

Funkcionalnost

Testiranje reakcije na nedostajući SendGrid API ključ.

Ispitni slučaj

Ulazni podaci:

- SEND_GRID_KEY environment varijabla: `null`
- toEmail: `test@example.com`
- subject: `"Test Subject"`
- message: `"Test Message Content"`

Očekivani rezultati:

- Exception s porukom: `"Null SendGridKey"`
- Email nije poslan

Dobiveni rezultati: ☒ PASS

Postupak provođenja ispitivanja

1. Postavljanje SEND_GRID_KEY na `null`
2. Kreiranje EmailSenderService instance
3. Poziv `SendEmailAsync` unutar `Assert.ThrowsAsync`
4. Provjera tipa iznimke (Exception)
5. Provjera poruke iznimke

```

[Test]
public void SendEmailAsync_WithNullApiKey_ThrowsException()
{
    Environment.SetEnvironmentVariable("SEND_GRID_KEY", null);

    var service = new EmailSenderService();

    var ex = Assert.ThrowsAsync(async () =>
    {
        await service.SendEmailAsync(TestEmail, TestSubject, TestMessage);
    });

    Assert.That(ex.Message, Is.EqualTo("Null SendGridKey"));
}

```

4.3 SendEmailAsync_WithEmptyApiKey_ThrowsException (Rubni uvjet)

Funkcionalnost

Testiranje reakcije na prazan SendGrid API ključ.

Ispitni slučaj

Ulazni podaci:

- SEND_GRID_KEY: `string.Empty`
- toEmail: `test@example.com`
- subject: `"Test Subject"`
- message: `"Test Message Content"`

Očekivani rezultati:

- Exception s porukom: `"Null SendGridKey"`

Dobiveni rezultati: ☐ PASS

Postupak provođenja ispitivanja

1. Postavljanje SEND_GRID_KEY na prazan string
2. Kreiranje service instance
3. Poziv metode i hvatanje iznimke
4. Provjera poruke iznimke (ista validacija kao za null)

```
[Test]

public void SendEmailAsync_WithEmptyApiKey_ThrowsException()
{
    Environment.SetEnvironmentVariable("SEND_GRID_KEY", string.Empty);

    var service = new EmailSenderService();

    var ex = Assert.ThrowsAsync(async () =>
        await service.SendEmailAsync(TestEmail, TestSubject, TestMessage));

    Assert.That(ex.Message, Is.EqualTo("Null SendGridKey"));
}
```

4.4 SendEmailAsync_WithValidParameters_AddsCorrectRecipient (Redovni slučaj)

Funkcionalnost

Testiranje ispravnog dodavanja primatelja emaila.

Ispitni slučaj

Ulazni podaci:

- toEmail: `test@example.com`
- subject: `"Test Subject"`
- message: `"Test Message Content"`

Očekivani rezultati:

- Personalizations lista nije prazna
- Prvi primatelj email: `test@example.com`

Dobiveni rezultati: ☐ PASS

Postupak provođenja ispitivanja

1. Kreiranje TestableEmailSenderService
2. Poziv `SendEmailAsync`
3. Provjera LastSentMessage.Personalizations:
 - Nije null
 - Count > 0
4. Ekstraktovanje prvog primatelja
5. Provjera email adrese primatelja

```
[Test]

public async Task SendEmailAsync_WithValidParameters_AddsCorrectRecipient()
{
    var service = new TestableEmailSenderService(HttpStatusCode.OK);
```

```

        await service.SendEmailAsync(TestEmail, TestSubject, TestMessage);

        Assert.That(service.LastSentMessage, Is.Not.Null);

        Assert.That(service.LastSentMessage.Personalizations, Is.Not.Null);

        Assert.That(service.LastSentMessage.Personalizations.Count, Is.GreaterThan(0));

        var recipient = service.LastSentMessage.Personalizations[0].Tos[0];

        Assert.That(recipient.Email, Is.EqualTo(TestEmail));
    }
}

```

4.5 SendEmailAsync_WithValidParameters_DisablesClickTracking (Redovni slučaj)

Funkcionalnost

Testiranje da je click tracking onemogućen u poslanom emailu.

Ispitni slučaj

Ulazni podaci:

- Standardni test parametri

Očekivani rezultati:

- TrackingSettings.ClickTracking.Enable: `false`

Dobiveni rezultati: ☒ PASS

Postupak provođenja ispitivanja

1. Poziv `SendEmailAsync`
2. Provjera `LastSentMessage.TrackingSettings`
3. Provjera `ClickTracking` nije null
4. Provjera da je `Enable` postavljen na `false`

```

[Test]
public async Task SendEmailAsync_WithValidParameters_DisablesClickTracking()
{
    var service = new TestableEmailSenderService(HttpStatusCode.OK);

    await service.SendEmailAsync(TestEmail, TestSubject, TestMessage);

    Assert.That(service.LastSentMessage, Is.Not.Null);

    Assert.That(service.LastSentMessage.TrackingSettings, Is.Not.Null);

    Assert.That(service.LastSentMessage.TrackingSettings.ClickTracking, Is.Not.Null);

    Assert.That(service.LastSentMessage.TrackingSettings.ClickTracking.Enable, Is.False);
}

```

4.6 SendEmailAsync_WhenSendGridReturnsError_ThrowsException (Izazivanje pogreške)

Funkcionalnost

Testiranje reakcije na različite HTTP error kodove od SendGrid API-ja.

Ispitni slučaj

Ulazni podaci:

- Response status: `BadRequest` / `Unauthorized` / `InternalServerError`

Očekivani rezultati:

- Exception s porukom jednaka status kodu

Dobiveni rezultati: ☒ PASS (3 test scenarija)

Postupak provođenja ispitivanja

Za svaki status kod (BadRequest, Unauthorized, InternalServerError):

1. Kreiranje TestableEmailSenderService s tim status kodom
2. Poziv `SendEmailAsync`
3. Provjera da se baca Exception
4. Provjera poruke iznimke odgovara status kodu

```
[Test]
```

```
public void SendEmailAsync_WhenSendGridReturnsError_ThrowsException()
{
    var service = new TestableEmailSenderService(HttpStatusCode.BadRequest);

    var ex = Assert.ThrowsAsync(async () =>
        await service.SendEmailAsync(TestEmail, TestSubject, TestMessage));

    Assert.That(ex.Message, Is.EqualTo(HttpStatusCode.BadRequest.ToString()));
}
```

```
[Test]
```

```
public void SendEmailAsync_WhenSendGridReturnsUnauthorized_ThrowsException()
{
    var service = new TestableEmailSenderService(HttpStatusCode.Unauthorized);

    var ex = Assert.ThrowsAsync(async () =>
        await service.SendEmailAsync(TestEmail, TestSubject, TestMessage));

    Assert.That(ex.Message, Is.EqualTo(HttpStatusCode.Unauthorized.ToString()));
}
```

```
[Test]
```

```
public void SendEmailAsync_WhenSendGridReturnsInternalServerError_ThrowsException()
{
    var service = new TestableEmailSenderService(HttpStatusCode.InternalServerError);

    var ex = Assert.ThrowsAsync(async () =>
        await service.SendEmailAsync(TestEmail, TestSubject, TestMessage));

    Assert.That(ex.Message, Is.EqualTo(HttpStatusCode.InternalServerError.ToString()));
}
```

4.7 SendEmailAsync_WithSpecialCharactersInMessage_SendsCorrectly (Rubni uvjet)

Funkcionalnost

Testiranje slanja emaila sa specijalnim znakovima i HTML tagovima.

Ispitni slučaj

Ulazni podaci:

- message: "Hello World! Special chars: @#\$\$%^&* () "

Očekivani rezultati:

- PlainTextContent sadrži cijelu poruku sa svim znakovima
- HtmlContent sadrži cijelu poruku sa svim znakovima

Dobiveni rezultati: ☐ PASS

Postupak provođenja ispitivanja

1. Definiranje poruke sa specijalnim znakovima i HTML tagovima
2. Poziv `SendEmailAsync`
3. Provjera da PlainTextContent i HtmlContent sadrže točnu poruku
4. Korištenje `Assert.Multiple` za obje provjere

```
[Test]

public async Task SendEmailAsync_WithSpecialCharactersInMessage_SendsCorrectly()
{
    var service = new TestableEmailSenderService(HttpStatusCode.OK);

    var messageWithSpecialChars = "Hello World! Special chars: @#$$%^&*()";

    await service.SendEmailAsync(TestEmail, TestSubject, messageWithSpecialChars);

    Assert.Multiple(() =>
    {
        Assert.That(service.LastSentMessage.PlainTextContent, Is.EqualTo(messageWithSpecialChars));

        Assert.That(service.LastSentMessage.HtmlContent, Is.EqualTo(messageWithSpecialChars));
    });
}
```

4.8 SendEmailAsync_WithMultipleCalls_SendsEachEmailIndependently (Redovni slučaj)

Funkcionalnost

Testiranje da višestruki pozivi metode šalju neovisne emailove.

Ispitni slučaj

Ulazni podaci:

- Poziv 1:
 - email: `user1@example.com`
 - subject: `"Subject 1"`
 - message: `"Message 1"`
- Poziv 2:
 - email: `user2@example.com`
 - subject: `"Subject 2"`
 - message: `"Message 2"`

Očekivani rezultati:

- Dvije različite SendGridMessage instance
- Svaki email ima svoje podatke

Dobiveni rezultati: ☐ PASS

Postupak provođenja ispitivanja

1. Prvi poziv `SendEmailAsync` s prvim setom podataka
2. Spremanje reference na firstMessage
3. Drugi poziv `SendEmailAsync` s drugim setom podataka
4. Spremanje reference na secondMessage
5. Provjera da firstMessage i secondMessage nisu ista instanca
6. Verifikacija da su poruke neovisne

```
[Test]

public async Task SendEmailAsync_WithMultipleCalls_SendsEachEmailIndependently()
{
    var service = new TestableEmailSenderService(HttpStatusCode.OK);

    var firstMessage = await service.SendEmailAsync(TestEmail, TestSubject, "Message 1");
    var secondMessage = await service.SendEmailAsync(TestEmail, TestSubject, "Message 2");

    Assert.That(firstMessage, Is.Not.EqualTo(secondMessage));
}
```

```
var service = new TestableEmailSenderService(HttpStatusCode.OK);

var email1 = "user1@example.com";

var email2 = "user2@example.com";

var subject1 = "Subject 1";

var subject2 = "Subject 2";

await service.SendEmailAsync(email1, subject1, "Message 1");

var firstMessage = service.LastSentMessage;

await service.SendEmailAsync(email2, subject2, "Message 2");

var secondMessage = service.LastSentMessage;

Assert.That(firstMessage, Is.Not.SameAs(secondMessage));

}
```

Sažetak Testnih Slučajeva

Ukupan broj testova: 18

Distribucija testova po kategorijama

Kategorija	Broj testova	Postotak
Redovni slučajevi	8	44.4%
Rubni uvjeti	6	33.3%
Izazivanje pogreške	7	38.9%
Nepostojeća funkcionalnost 2	11.1%	

Pokrivenost po kategorijama

1. Redovni slučajevi: 8 testova

#	Test	Komponenta	Status
1	Register_WithNewUser_ReturnsOk	AuthController	❑ PASS
2	Login_WithValidCredentials_ReturnsOkWithToken	AuthController	❑ PASS
3	ExternalLoginCallback_WithExistingUser_ReturnsRedirectWithToken	AuthController	❑ PASS
4	ExternalLoginCallback_WithNewUser_CreatesUserAndReturnsRedirect	AuthController	❑ PASS
5	SendEmailAsync_WithValidParameters_SendsEmailSuccessfully	EmailSenderService	❑ PASS
6	SendEmailAsync_WithValidParameters_AddsCorrectRecipient	EmailSenderService	❑ PASS
7	SendEmailAsync_WithValidParameters_DisablesClickTracking	EmailSenderService	❑ PASS
8	SendEmailAsync_WithMultipleCalls_SendsEachEmailIndependently	EmailSenderService	❑ PASS

2. Rubni uvjeti: 6 testova

#	Test	Komponenta	Status
1	Register_WithExistingEmail_ReturnsBadRequest	AuthController	❑ PASS
2	Login_WithInvalidEmail_ReturnsUnauthorized	AuthController	❑ PASS
3	Login_WithInvalidPassword_ReturnsUnauthorized	AuthController	❑ PASS
4	ExternalLoginCallback_WithNoEmailClaim_ReturnsBadRequest	AuthController	❑ PASS
5	SendEmailAsync_WithEmptyApiKey_ThrowsException	EmailSenderService	❑ PASS
6	SendEmailAsync_WithSpecialCharactersInMessage_SendsCorrectly	EmailSenderService	❑ PASS

3. Izazivanje pogreške: 7 testova

#	Test	Komponenta	Status
1	Register_WhenUserCreationFails_ReturnsBadRequest	AuthController	❑ PASS
2	Register_WhenAddToRoleFails_ReturnsBadRequest	AuthController	❑ PASS
3	ExternalLoginCallback_WithRemoteError_ReturnsBadRequest	AuthController	❑ PASS
4	SendEmailAsync_WithNullApiKey_ThrowsException	EmailSenderService	❑ PASS
5	SendEmailAsync_WhenSendGridReturnsError_ThrowsException (BadRequest)	EmailSenderService	❑ PASS
6	SendEmailAsync_WhenSendGridReturnsUnauthorized_ThrowsException	EmailSenderService	❑ PASS
7	SendEmailAsync_WhenSendGridReturnsInternalServerError_ThrowsException	EmailSenderService	❑ PASS

4. Testiranje nepostojeće/neispravne funkcionalnosti: 2 testa

#	Test	Komponenta	Status
1	ExternalLoginCallback_WithNoExternalLoginInfo_ReturnsBadRequest	AuthController	❑ PASS
2	ExternalLoginCallback_WithNoEmailClaim_ReturnsBadRequest	AuthController	❑ PASS

Tehnička Dokumentacija

Korištene tehnologije i alati

Testing Framework

- **NUnit 4.2.2** - Unit testing framework za .NET
- **NUnit.Analyzers 4.4.0** - Statička analiza test koda

Mocking Libraries

- **Moq 4.20.72** - Framework za stvaranje mock objekata
 - Korišteno za mockiranje UserManager, SignInManager, IMapper, IConfiguration

Dependencije

- ****ASP.NET Core Identity **** - Sustav za autentifikaciju i autorizaciju
- **AutoMapper** - Object-to-object mapping
- **SendGrid** - Email delivery service
- **System.IdentityModel.Tokens.Jwt** - JWT token generiranje i validacija

Setup i Konfiguracija

AuthControllerTests - SetUp metoda

```
[SetUp]

public void Setup()

{

    _configurationMock = new Mock();

    _mapperMock = new Mock();

    var userStoreMock = new Mock<IUserStore>();

    _userManagerMock = new Mock<UserManager>(

        userStoreMock.Object, null, null, null, null, null, null, null, null);

    var contextAccessorMock = new Mock();

    var claimsFactoryMock = new Mock<IUserClaimsPrincipalFactory>();

    _signInManagerMock = new Mock<SignInManager>(

        _userManagerMock.Object,

        contextAccessorMock.Object,

        claimsFactoryMock.Object,

        null, null, null, null);

    SetupJwtConfiguration();

    _controller = new AuthController(

        _configurationMock.Object,

        _mapperMock.Object,

        _signInManagerMock.Object,

        _userManagerMock.Object);
}
```

Ključne komponente:

- Kreiranje mock objekata za sve ovisnosti

- Konfiguracija JWT postavki
- Instanciranje kontrolera s mockiranim ovisnostima

EmailSenderServiceTests - SetUp metoda

```
[SetUp]

public void SetUp()

{

    Environment.SetEnvironmentVariable("SEND_GRID_KEY", TestApiKey);

}
```

Ključne komponente:

- Postavljanje environment varijable za SendGrid API ključ
- Jednostavna inicijalizacija zbog manje ovisnosti

TearDown metode

```
[TearDown]

public void TearDown()

{

    Environment.SetEnvironmentVariable("SEND_GRID_KEY", null);

    // ili

    Environment.SetEnvironmentVariable("JWT_KEY", null);

}
```

Svrha:

- Čišćenje environment varijabli nakon svakog testa
- Osiguravanje izolacije između testova
- Sprječavanje međusobnog utjecaja testova

Mock Objekti i Njihova Uporaba

1. UserManager Mock

```
var userStoreMock = new Mock<IUserStore>();

_userManagerMock = new Mock<UserManager>(

    userStoreMock.Object, null, null, null, null, null, null, null, null, null);
```

Najčešće mockane metode:

- FindByEmailAsync() - Pronalaženje korisnika po emailu
- CreateAsync() - Kreiranje novog korisnika
- AddToRoleAsync() - Dodavanje korisnika u ulogu
- FindByLoginAsync() - Pronalaženje korisnika po vanjskom login provideru
- AddLoginAsync() - Povezivanje vanjskog logina s korisnikom

2. SignInManager Mock

```
_signInManagerMock = new Mock<SignInManager>(

    _userManagerMock.Object,

    contextAccessorMock.Object,

    claimsFactoryMock.Object,

    null, null, null, null);
```

Najčešće mockane metode:

- CheckPasswordSignInAsync() - Provjera lozinke pri prijavi
- GetExternalLoginInfoAsync() - Dohvaćanje informacija o vanjskom loginu

3. IMapper Mock

```
_mapperMock = new Mock();
```

```
_mapperMock.Setup(x => x.Map(registerDto)).Returns(user);
```

Svrha:

- Mapiranje DTO objekata u domenski modele
 - Izolacija AutoMapper logike od testova
-

4. IConfiguration Mock

```
var jwtSectionMock = new Mock();
```

```
jwtSectionMock.Setup(x => x["Issuer"]).Returns("TestIssuer");
```

```
jwtSectionMock.Setup(x => x["Audience"]).Returns("TestAudience");
```

```
_configurationMock.Setup(x => x.GetSection("Jwt")).Returns(jwtSectionMock.Object);
```

Svrha:

- Konfiguracija JWT postavki za testiranje
 - Postavljanje test vrijednosti za Issuer i Audience
-

TestableEmailSenderService - Helper Klasa

```
public class TestableEmailSenderService : EmailSenderService
```

```
{
```

```
    private readonly HttpStatusCode _responseStatusCode;
```

```
    public SendGridMessage LastSentMessage { get; private set; }
```

```
    public TestableEmailSenderService(HttpStatusCode responseStatusCode)
```

```
    {
```

```
        _responseStatusCode = responseStatusCode;
```

```
    }
```

```
    public new async Task SendEmailAsync(string toEmail, string subject, string message)
```

```
    {
```

```
        var apiKey = Environment.GetEnvironmentVariable("SEND_GRID_KEY");
```

```
        if (string.IsNullOrEmpty(apiKey))
```

```
        {
```

```
            throw new Exception("Null SendGridKey");
```

```
        }
```

```
        await ExecuteTestable(apiKey, subject, message, toEmail);
```

```
    }
```

```
    private async Task ExecuteTestable(string apiKey, string subject, string message, string toEmail)
```

```
    {
```

```
        var msg = new SendGridMessage
```

```
        {
```

```
            From = new EmailAddress("support@progimindfulness.app", "Support"),
```

```
            Subject = subject,
```

```
            PlainTextContent = message,
```

```
            HtmlContent = message
```

```

    };

    msg.AddTo(new EmailAddress(toEmail));

    msg.SetClickTracking(false, false);

    LastSentMessage = msg;

    if (_responseStatusCode != HttpStatusCode.OK &&
        _responseStatusCode != HttpStatusCode.Accepted)
    {
        throw new Exception(_responseStatusCode.ToString());
    }

    await Task.CompletedTask;
}
}
}

```

Ključne značajke:

- Nasljeđuje `EmailSenderService` za testiranje
- Omogućava simulaciju različitih HTTP status kodova
- Sprema posljednju poruku za provjeru u testovima
- Ne šalje stvarne emailove (izolacija)

Pokretanje Testova

Putem Visual Studio

1. Otvorite Test Explorer: `Test > Test Explorer`
2. Kliknite `Run All` za pokretanje svih testova
3. Pregledajte rezultate u Test Explorer prozoru

Putem .NET CLI

```

# Pokretanje svih testova

dotnet test

# Pokretanje testova s detaljnim outputom

dotnet test --verbosity detailed

# Pokretanje testova s code coverage

dotnet test /p:CollectCoverage=true

# Pokretanje specifične test klase

dotnet test --filter "FullyQualifiedName~Tests"

# Pokretanje specifičnog testa

dotnet test --filter "FullyQualifiedName~Register_WithNewUser_ReturnsOk"

```

Očekivani Output

```
Passed! - Failed: 0, Passed: 18, Skipped: 0, Total: 18, Duration: < 1 s
```

Git Repozitorij

Struktura Repozitorija

```
Mindfulness/  
├─ Mindfulness.Server/  
│   └─ Controllers/  
│       └─ AuthController.cs  
│       └─ Services/  
│           └─ EmailSenderService.cs  
│           └─ Models/  
│               └─ User.cs  
│               └─ Dtos/  
│                   └─ User/  
│                       └─ UserRegisterDto.cs  
│                       └─ UserLoginDto.cs  
│  
└─ Mindfulness.Tests/  
    └─ Tests.cs  
    └─ EmailSenderServiceTests.cs  
    └─ Mindfulness.Tests.csproj
```

Dostupnost Izvornog Koda

Izvorni kodovi testova dostupni su u Git repozitoriju:

Datoteke:

- Mindfulness.Tests/AuthControllerTests.cs
 - AuthController Register testovi (4 testa)
 - AuthController Login testovi (3 testa)
 - AuthController ExternalLoginCallback testovi (5 testova)
- Mindfulness.Tests/EmailSenderServiceTests.cs
 - EmailSenderService testovi (11 testova)
 - TestableEmailSenderService helper klasa

**Ispitivanje sustava **

Cilj ispitivanja sustava je testiranje ponašanja cijelog sustava u uvjetima stvarnog korištenja, uz posebnu pažnju na međusobnu povezanost svih komponenti. Ispitivanje treba obuhvatiti sve aspekte sustava i njegovu interakciju s korisnicima.

Zadaci:

Razviti minimalno **4 ispitna slučaja** koji obuhvaćaju:

- Redovne slučajeve:** očekivano ponašanje sustava.
- Rubne uvjete:** reakcija sustava na granične ulaze.
- Poziv nepostojećih funkcionalnosti:** testiranje kako sustav reagira na neimplementirane ili neispravne funkcije.

Struktura ispitnih slučajeva za Selenium:

- Ulazi:**
 - Konkretni podaci koji se unose u sustav (npr. korisničko ime i lozinka za prijavu).
 - Simulacija korisničkih akcija (klikanje, unos teksta, navigacija).
- Koraci ispitivanja:**
 - Npr. Detaljan opis koraka koje Selenium izvršava:
 - Otvoriti aplikaciju u pregledniku.
 - Unijeti podatke u formu.
 - Kliknuti na gumb za potvrdu.

4. Verificirati očekivane rezultate (npr. prijava uspješna).

3. **Očekivani izlaz:**

- Očekivani rezultat ispitivanja (npr. korisnik preusmjeren na početnu stranicu nakon prijave).

4. **Dobiveni izlaz:**

- Priložiti logove, screenshotove ili izvještaje s generiranim rezultatima (npr. iz JUnit-a ili Selenija).

Alati za ispitivanje sustava:

- **Selenium IDE** ili prikladan obzirom na vaše razvojno okruženje:
 - Jednostavan alat za snimanje korisničkih akcija u pregledniku i automatsko ponavljanje testova.
 - Preporučuje se za osnovne ispitne slučajeve.
- **Selenium WebDriver:**
 - Omogućuje pisanje naprednih testova u različitim programskim jezicima (Java, Python, C#).
 - Preporučuje se za složenije testove, koji zahtijevaju detaljno prilagodbu i automatizaciju.

Primjeri ispitivanja Seleniomom:

1. **Formular za prijavu:**

u dokumentaciji obavezna su specifičnija ispitivanja vaše aplikacije! - **Ulaz:** Korisničko ime = "user@fer.ugnz.hr", Lozinka = "password123".

```
- **Koraci**:
```

```
1. Otvoriti aplikaciju.
```

```
2. Unijeti korisničko ime i lozinku.
```

```
3. Kliknuti na "Prijava".
```

```
4. Provjeriti je li korisnik preusmjeren na početnu stranicu.
```

```
- **Očekivani izlaz**:
```

```
"Prijava uspješna."
```

2. **Rubni uvjet – nevažeća lozinka:**

- **Ulaz:** Korisničko ime = "user@example.com", Lozinka = "malimedo".
- **Koraci:**
 1. Unijeti podatke u formu za prijavu.
 2. Kliknuti na "Prijava".
 3. Provjeriti je li prikazana poruka o grešci.
- **Očekivani izlaz:** "Pogrešno korisničko ime ili lozinka."

Prezentacija rezultata

Za oba tipa ispitivanja (komponenti i sustava) potrebno je:

- Jasno dokumentirati ulaze, korake, očekivane i dobivene rezultate.
- Priložiti slike ekrana, logove ili izvješća generirana alatima (npr. JUnit ili Selenium).
- Detaljno opisati ponašanje sustava, posebno u rubnim uvjetima.
- Navesti broj otkrivenih grešaka!

Korištene tehnologije i alati

Cilj: Jasno i precizno opisati tehnologije korištene u projektu kako bi se olakšalo održavanje, proširenje i suradnja u timu. Uključite informacije:

1. **Programski jezici:**

- C# 13
- TypeScript

2. **Radni okviri i biblioteke:**

- React 19.2.0
- ASP.NET Core 9.0

3. **Baza podataka:** Navesti vrstu baze (npr. PostgreSQL 13).

- PostgreSQL 18

4. **Razvojni alati:**

- JetBrains Rider 2025.2.3
- VSCode
- Astah UML
- Figma
- ERDplus

5. **Alati za ispitivanje:**

- NUnit 4.2
- Swagger UI

6. **Alati za razmještaj:**

- Docker 28.5.1
- Render

7. **Cloud platforma:**

- Render

Opis

Tehnologije korištene za izradu aplikacije

Za frontend smo koristili React i Typescript. Za uspostavu autentifikacije korisnika upotrijebili smo JWT. Za backend smo koristili JetBrains Rider, C# i ASP.NET 9, open-source web framework. Backend i frontend smo kontenjerizirali s dockerom te smo za deployment iskoristili platformu Render. Bazu podataka idejno smo ostvarili aplikacijom ERDplus te je realizirali pomoću PostgreSQL-a, sustava za upravljanje open-source bazama.

Tehnologije korištene za izradu dokumentacije

Dokumentacija je pisana na Wiki dijelu GitHub-a. Za modeliranje dijagrama obrazaca uporabe, sekvencijskih dijagrama, dijagrama komponenti te klasnih dijagrama koristili smo Astah UML, aplikaciju za izradu dijagrama i grafova.

Ovaj odjeljak dokumentacije treba dati detaljne smjernice za instalaciju, konfiguraciju, pokretanje i administraciju aplikacije. Cilj je olakšati postavljanje aplikacije na razvojnom, ispitnom i produkcijskom okruženju.

1. Instalacija

Ovdje treba navesti korake potrebne za instalaciju svih potrebnih komponenti:

- **Preduvjeti:**

- .NET 9
- Node.js 24
- Docker 28.5.1 (samo za deploy)
- Entity Framework Core tools 9.0.10

- **Preuzimanje:**

```
git clone https://github.com/FPilica/Programsko-inzenjerstvo.git
```

```
cd Programsko-inzenjerstvo
```

Instalacija ovisnosti: Upute za instaliranje ovisnosti.

```
cd mindfulness.client && npm install && cd ..
```

2. Postavke

Detaljne upute za konfiguraciju aplikacije:

- **Konfiguracijske datoteke:**

- Potrebno je u appsettings.json promijeniti client id za vanjske OAuth2.0 servise:

```
"Google": {  
  
    "ClientId": "{ClientId Google OAuth2.0 servisa}"  
  
},  
  
"Microsoft": {  
  
    "ClientId": "{ClientId Microsoft OAuth2.0 servisa}"  
  
}
```

- Nakon toga potrebno je stavitiConnectionString za bazu podataka isto u appsettings.json (samo potrebno za deployment, u razvojnom okruženju se koristi baza u memoriji):

```
"ConnectionStrings": {
```

```
"Database": "{ConnectionString za spajanje na PostgreSQL bazu podataka}"
```

```
}
```

- Kako bi u .NET User Secrets spremili sve potrebne tajne, potrebno je izvršiti sljedeće naredbe:

```
dotnet user-secrets set "JwtKey" "{ključ po izboru}"
```

```
dotnet user-secrets set "Authentication:Google:ClientSecret" "{ClientSecret Google OAuth2.0 servisa}"
```

```
dotnet user-secrets set "Authentication:Microsoft:ClientSecret" "{ClientSecret Microsoft OAuth2.0 servisa}"
```

```
dotnet user-secrets set "SendGrid:Key" "{API Key SendGrid servisa}"
```

- **Postavke baze podataka:** Upute za inicijalizaciju baze podataka, uključujući migracije i postavljanje inicijalnih podataka.

```
dotnet ef migrations add {Ime Migracije} --project ./Mindfulness.Server
```

```
dotnet ef database update --project ./Mindfulness.Server
```

3. Pokretanje aplikacije

Upute za pokretanje aplikacije:

```
git checkout -b dev --track origin/dev
```

```
dotnet run --project ./Mindfulness.Server
```

- Automatski će se pokrenuti i frontend u komandnom prozoru, gdje će pisati URL stranice

- **Provjera rada:**

```
https://localhost:60665/
```

4. Upute za administratore

Smjernice za administratore aplikacije nakon puštanja u pogon:

- **Pristup administratorskom sučelju:**

- Početni podaci za prijavu:

- Email: admin@admin.com

- Password: admin1!A

- Dovoljno je ulogirati se i dostupne su sve mogućnosti Admin uloge.

- **Pristup administratorskom sučelju:**

- Početni podaci za prijavu:

- Email: coach@coach.com

- Password: coach1!C

- Dovoljno je ulogirati se i dostupne su sve mogućnosti Coach uloge.

- **Redovito održavanje:**

- Pregled logova.

- Povlačenje novih verzija iz Git repozitorija i ponovno pokretanje aplikacije.

```
git pull origin main
```

```
npm install
```

```
npm run build
```

```
npm start
```

- **Rješavanje problema:** Logovi su dostupni na stranici hosting provider-a Render.

Opis prisutpa aplikaciji na javnom poslužitelju

Pristup aplikaciji Aplikacija je dostupna na:

<https://mindfulnessfrontend.onrender.com>

- Pri pristupanju stranici, prvi request na backend će kasniti ili se neće izvršiti ispravno. Preporučljivo je za prvi request poslati neki osnovni login request, pa tek kada stranica vrati grešku, sve dalje će normalno raditi

- **Pristup administratorskom sučelju:**

- Početni podaci za prijavu:

- Email: admin@admin.com

- Password: admin1!A

- Dovoljno je ulogirati se i dostupne su sve mogućnosti Admin uloge.

- **Pristup administratorskom sučelju:**

- Početni podaci za prijavu:
 - Email: coach@coach.com
 - Password: coach1!C
- Dovoljno je ulogirati se i dostupne su sve mogućnosti Coach uloge.

Naš zadatak bio je razviti aplikaciju koja korisnicima pruža podršku pri praćenju mentalnog zdravlja i prakticiranju mindfulnessa.

U prvoj smo fazi okupili tim od 6 ljudi i podijelili se na grupu koja radi backend dio aplikacije, grupu koja radi frontend dio aplikacije i voditelja. Za početak je dokumentacija bila ključna, tako da smo se posvetili izradi vizualnih rješenja aplikacije, pisanju obrazaca uporabe i funkcionalnih zahtjeva, izradi početnih dijagrama, detaljima ponašanja aplikacije, itd.

U drugoj fazi fokusirali smo se na razvoj same aplikacije u JatBrains Rider i React razvojnim okruženjima te krenuli izrađivati popis zadataka na aplikaciji Jira, koja nam je omogućila direktan uvid u količinu zadataka te nam olakšala raspodjelu posla. Dokumentirali smo ostatak potrebnih dijagrama kako bi funkcionalnosti bile jasnije budućim korisnicima. Komunikacija je bila ključna jer smo u ovoj fazi mnogo ovisili o ostalim članovima tima i njihovom radu. U ovoj smo fazi naišli na neke probleme u komunikaciji, ali smo ih uspješno riješili.

Najviše vremena nam je oduzelo upoznavanje s alatima potrebnima za izradu naše aplikacije, ali ti problemi su prevladani kvalitetnom komunikacijom putem WhatsAppa i Discorda.

Također smo naišli na probleme s iskustvom. Naime, kako bismo projekt realizirali brže i kvalitetnije, posebno bi bilo korisno dodatno znanje iz područja upravljanja projektima, posebno u smislu bolje procjene potrebnog vremena i resursa. Osim navedenog stekli smo znanje te što je potrebno za izradu i implementaciju web aplikacije.

Neimplementirani funkcionalni zahtjevi

F-007, F-008, NF - 1.1.3 (zbog Rendera), NF - 1.5.2

i na kvadrat

Rev. Opis promjene/dodatka	Autori	Datum
0.1 Napravljen Github, kostur projekta, README, Licenca	Fran Pilica	10.10.2025.
0.2 Napravljen wiki page	Ivan Lukić	22.10.2025.
0.3 Napravljen wiki predložak	Lovre Baus	22.10.2025
0.4 Započet README	Lovre Baus	24.10.2025.
0.5 Napisani funkcijski i nefunkcijski zahtjevi	Lovre Baus	24.10.2025.
0.6 Započet prikaz aktivnosti grupe	Lovre Baus	24.10.2025.
0.7 Započet dnevnik promjene dokumentacije	Lovre Baus	24.10.2025.
0.8 Napravljen ERD model baze podataka	Ana Smoljo	31.10.2025.
0.9 Skoro dovršen opis projektnog zadatka	Ana Smoljo	2.11.2025.
1.0 Nadopunjena analiza zahtjeva sa dionicima i aktorima	Lovre Baus	6.11.2025.
1.1 Nadopunjena aktivnost grupe	Lovre Baus	6.11.2025.
1.2 Napisana arhitektura sustava	Ana Smoljo	12.11.2025.
1.3 Razrađena baza podataka, napravljen dijagram baze podataka	Ana Smoljo	12.11.2025.
1.4 Napravljen većina dijagrama obrazaca uporabe	Fran Pilica	12.11.2025.
1.5 Napisan opis obrazaca uporabe	Lovre Baus	12.11.2025.
1.6 Napravljen sekvencijski dijagram za registraciju i prijavu	Ana Smoljo	12.11.2025
1.7 Napremljene tehnologije za implementaciju aplikacije	Ana Smoljo	112.11.2025
1.8 Napravljen jedan UML dijagram aktivnosti	Karlo Ivančić	12.11.2025.
1.9 Napravljen dijagram obrazaca uporabe za korisničke uloge	Ivan Lukić	12.11.2025.
2.0 Napravljen dijagram obrazaca uporabe za osnovne poslovne procese	Elizej Kolić	13.11.2025.
2.1 Napravljen sekvencijski dijagram za korištenje kalendara	Karlo Ivančić	13.11.2025.
2.2 Napisana provjera uključenosti ključnih funkcionalnosti u obrasce uporabe, doraden opis obrazaca uporabe	Lovre Baus	13.11.2025.
2.3 Napisane upute za deployment	Fran Pilica	13.11.2025
2.4 Popravljen grupacija nefunkcijskih zahtjeva	Ana Smoljo	20.1.2026
2.5 Ispravljene gramatičke pogreške	Ana Smoljo	20.1.2026
2.6 Dodani sekvencijski dijagrami i opisi istih	Ana Smoljo	20.1.2026
2.7 Detaljizacija arhitekture sustava	Karlo Ivančić	21.1.2026
2.8 Izmijenjene aktivnosti	Ana Smoljo	23.1.2026
2.9 Ažurirane upute za puštanje u pogon	Fran Pilica	23.1.2026

Kontinuirano osvježavanje

1. Programsko inženjerstvo, FER ZEMRIS, <http://www.fer.hr/predmet/proinz>
2. The Unified Modeling Language, <https://www.uml-diagrams.org/>
3. Astah Community, <http://astah.net/editions/uml-new>
4. Stack Overflow, <https://stackoverflow.com/questions>
5. Render, <https://render.com/>

1. sastanak

- **Datum:** 10. listopada 2025.
- **Prisustvovali** F.Pilica, L.Baus, A.Smoljo, I.Lukić, K.Ivančić, E.Kojić
- **Teme sastanka:**
 - upoznavanje članova
 - raspodjela članova tima na backend i frontend
 - odabir voditelja
 - uspostavljena WhatsApp i Discord grupa za komunikaciju svih članova

2. sastanak

- **Datum:** 10. listopada 2025.
- **Prisustvovali:** F.Pilica, L.Baus, A.Smoljo, I.Lukić, K.Ivančić, E.Kojić
- **Teme sastanka:**
 - sastanak s asistentom i demosom - uputili nas bolje u zadatak
 - uspostava GitHuba

3. sastanak

- **Datum:** 17. listopada 2025.
- **Prisustvovali:** F.Pilica, L.Baus, A.Smoljo, I.Lukić, K.Ivančić, E.Kojić
- **Teme sastanka:**
 - online sastanak sa asistentom
 - definiranje funkcijskih i nefunkcijskih zahtjeva

4. sastanak

- **Datum:** 24. listopada 2025.
- **Prisustvovali:** F.Pilica, L.Baus, A.Smoljo, I.Lukić, K.Ivančić
- **Teme sastanka:**
 - sastanak sa asistentom i demosom
 - prezentiranje dosadašnjeg rada
 - prolazak kroz funkcionalne i ne funkcionalne zahtjeve
 - dogovoreno da ubrzo krenemo sa radom na implementaciji

5. sastanak

- **Datum:** 31. listopada 2025.
- **Prisustvovali:** F.Pilica, L.Baus, A.Smoljo, I.Lukić, K.Ivančić
- **Teme sastanka:**
 - spajanje frontenda i backenda
 - Login i Register stranice

6. sastanak

- **Datum:** 14. studenog 2025.
- **Prisustvovali:** F.Pilica, L.Baus, A.Smoljo, I.Lukić, K.Ivančić
- **Teme sastanka:**
 - dogovor oko postavljanja finalne verzije dokumentacije
 - dogovor oko promjene dodatne funkcionalnosti

7. sastanak

- **Datum:** 12. prosinca 2025.
- **Prisustvovali:** F.Pilica, L.Baus, A.Smoljo, I.Lukić, K.Ivančić
- **Teme sastanka:**
 - sastanak s asistentom i demosom
 - prikaz napretka i statusa projekta

8. sastanak

- **Datum:** 19. prosinca 2025.

- **Prisustvovali:** F.Pilica, L.Baus, A.Smoljo, I.Lukić, K.Ivančić
- **Teme sastanka:**
 - sastanak s asistentom i demosom
 - prikaz napretka i statusa projekta

9. sastanak

- **Datum:** 9. siječnja 2026.
- **Prisustvovali:** F.Pilica, L.Baus, A.Smoljo, I.Lukić, K.Ivančić
- **Teme sastanka:**
 - prikaz alpha verzije aplikacije
 - dogovor oko završetka i opsega funkcionalnosti

10. sastanak

- **Datum:** 16. siječnja 2026.
- **Prisustvovali:** F.Pilica, L.Baus, A.Smoljo, I.Lukić, K.Ivančić, E.Kojić
- **Teme sastanka:**
 - prikaz napretka alpha verzije
 - dogovor oko završnih detalja i konačne upute

11. sastanak

- **Datum:** 21. siječnja 2026.
- **Prisustvovali:** F.Pilica, L.Baus, A.Smoljo, I.Lukić, K.Ivančić, E.Kojić
- **Teme sastanka:**
 - dogovor oko testiranja i popravljnja nađenih pogrešaka
 - raspodjela finalnih zadataka vezanih uz dokumentaciju

Plan rada

Tjedan	Aktivnost na projektu	Angažirani članovi
3	uspostava komunikacije među članovima tima, uspostava GitHub repozitorija, definiranje funkcijskih i nefunkcijskih zahtjeva	FP, LB, KI, EK, IL, AS
4	razrada UML i sekvencijskih dijagrama, započet dizajn aplikacije, rad na dokumentaciji	FP, LB, KI, IL, AS
5	dizajniranje aplikacije, login i register stranice, dodana autentifikacija korisnika	FP, LB, KI, IL, AS
6	rad na dokumentaciji, povezivanje baze s aplikacijom	FP, LB, KI, EK, IL, AS
7	dogovorena kolektivna pauza od ispita	
8	dogovor o nastavku razvoja, rad na izgledu stranice sadržaja i prikladnih kontrolera	LB, AS, FP
9	rad na kalendaru s događajima	LB, AS, KI, FP
10	razrađeni dashboard i uvodni upitnik	FP, LB, IL, AS
11	dodane role, mogućnost filtriranja sadržaja i uređivanje profila	FP, LB, KI, IL, AS
12	dodane statistike i daily check-in, rad na dokumentaciji	FP, LB, KI, EK, IL, AS

Tablica aktivnosti

Zadatak	Fran Pilica Lovre Baus Karlo Ivančić Elizej Kojić Ivan Lukić Ana Smoljo					
Upravljanje projektom	5					
Opis projektnog zadatka						5
Funkcionalni zahtjevi		6				1
Opis pojedinih obrazaca		7				1
Dijagram obrazaca	5			2		
Sekvencijski dijagrami		1	1	3		4
Opis ostalih zahtjeva		1				
Arhitektura i dizajn sustava						6
Baza podataka			1			8
Dijagram razreda	1					1
Dijagram stanja			1.5			
Dijagram aktivnosti			1.5			
Dijagram komponenti						3
Korištene tehnologije i alati	1					1
Ispitivanje programskog rješenja		1			1	
Dijagram razmještaja						
Upute za puštanje u pogon	2					
Dnevnik sastajanja		0.5				3
Zaključak i budući rad						1
Popis literature						1
Istraživanje tehnologija i učenje	5	13	14	13	17	12

Zadatak	Fran	Pilica	Lovre	Baus	Karlo	Ivančič	Elizej	Kojić	Ivan	Lukić	Ana	Smoljo
Dizajn						15					1	
Frontend						99		27	31			
Backend		45			20						30	
Puštanje aplikacije u pogon		10										

Dijagram pregleda promjena



Ključni izazovi i rješenja

- Opis izazova: Najveći problem bio je smanjena angažiranost nekih članova tima. Također, nailazili smo na probleme sa spajanjem frontend-a i backend-a.
- Rješenja: Smanjenu angažiranost riješili smo timskim razgovorom i dogovorom o budućoj angažiranosti kako bi svi znali što očekivati. Probleme sa spajanjem riješili smo zajedničkim radom i testiranjem.